SEMANA 11 · AUTOMATIZACIÓN DE INFRAESTRUCTURA (IAC) — REPASO U2

Por qué automatizar la infraestructura

Parte 1 de 3 · Unidad 2

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 11 · Periodo 2026-2 · Barranquilla, Colombia

Consola, scripts e infraestructura como código: qué cambia cuando el plano de la nube se vuelve un archivo.

Tres entornos "iguales" que no lo son

El problema de la infraestructura manual no es el clic: es que nadie puede repetir exactamente lo que hizo.

DESARROLLO	PRUEBAS	PRODUCCIÓN
Creado por Ana, un lunes a mano, desde la consola	**Creado por Luis, de memoria** a mano, desde la consola	**Creado por un contratista** a mano, desde la consola
Parches al día	**Parches** **a medias**	**Parches** **atrasados**
Puerto 22 **abierto a todo Internet**	**Puerto 22** solo la oficina	**Puerto 22** solo la oficina
Tamaño t3.small	**Tamaño** t3.medium	**Tamaño** m5.large
Respaldo **ninguno**	**Respaldo** **ninguno**	**Respaldo** diario

Los tres se crearon "igual", pero no lo son.

Lo que funciona en pruebas falla en producción, y nadie sabe exactamente por qué.

3 entornos, 3 realidades

Fuente: elaboración propia (caso Corporación Industrial del Caribe); concepto de deriva en AWS Well-Architected Framework

MAPA DE LA SEMANA

Tres partes para automatizar y repasar

Hoy cierra la Unidad 2: aprendemos a construir con código todo lo que ya sabíamos construir con clics.

Una hora de clase = tres videos de unos 15 minutos

1



Por qué automatizar

Consola, scripts e IaC
Imperativo y declarativo
Idempotencia y deriva

tema de la clase

2



IaC en la práctica

CloudFormation: pila,
conjuntos de cambios, deriva
Terraform y Bicep

tema de la clase

3



Entrega continua

Git, pipelines CI/CD
Entornos dev, test, prod
Pruebas y GitOps

tema de la clase

4



Repaso U2 y lab

Mapa de S07 a S10
Arquitectura de referencia
Lab IaC y Parcial 2

tema de la clase



Lab de automatización (IaC): 10 % de la nota

Se construye con la Parte 2 y la Parte 3.



Semana del Parcial 2: 20 % de rúbrica

Cubre las semanas 7 a 11 de la Unidad 2.

Fuente: calendario del curso 2026-2; equivalente a AWS Academy Cloud Architecting, módulo 11

Tres formas de crear infraestructura

La consola gana en la primera vez; el código gana desde la segunda.

CONSOLA

Clic a clic



Repetible



Auditable



Rápida la primera vez



Escala a 50 entornos



SCRIPTS

Comandos en secuencia



Repetible



Auditable



Rápida la primera vez



Escala a 50 entornos



IaC

Plantilla declarativa



Repetible



Auditable



Rápida la primera vez



Escala a 50 entornos



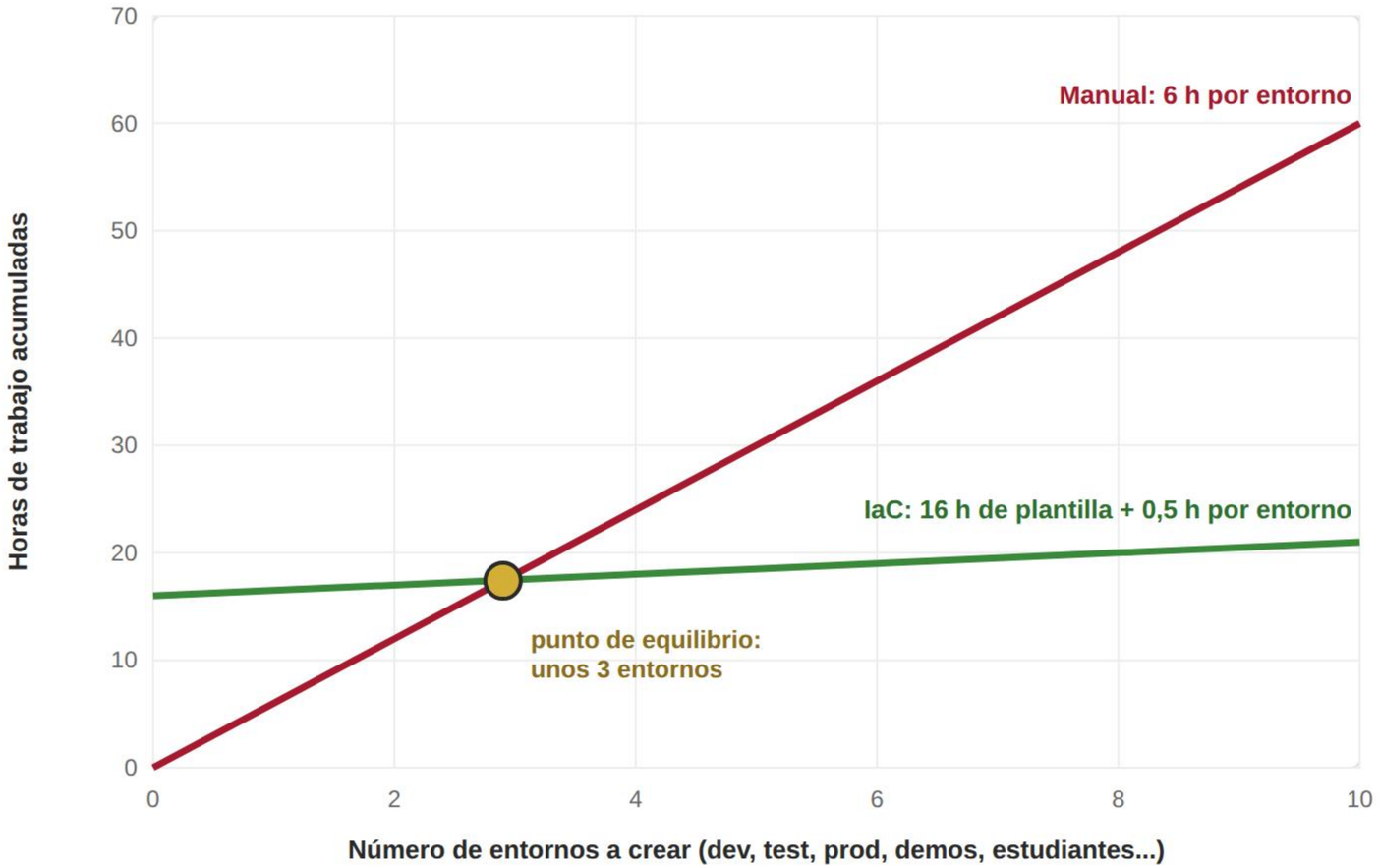
Valoración cualitativa: más puntos llenos significa mejor

Fuente: elaboración propia a partir de AWS Well-Architected Framework, pilar de excelencia operativa

DESARROLLO

El costo oculto del clic

Armar dos entornos a mano parece barato; armar diez revela el costo real.



Cifras ilustrativas para mostrar la forma de las curvas, no mediciones reales.

Cada clic es una oportunidad de error

Un entorno armado a mano rara vez es igual al anterior.

El primer día cuesta más

Escribir la plantilla es una inversión que se paga rápido.

Después, casi gratis

El entorno 10 cuesta lo mismo que el entorno 2: segundos de ejecución.

Fuente: elaboración propia (modelo ilustrativo); idea de "operaciones como código" de AWS Well-Architected Framework

Imperativo frente a declarativo

Imperativo es decir cómo hacerlo; declarativo es decir cómo debe quedar.

IMPERATIVO: la receta paso a paso

1. Crear la red virtual
2. Crear la subred pública
3. Crear el grupo de seguridad
4. Lanzar dos instancias
5. Registrarlas en el balanceador



Tú decides el orden.
Si el paso 4 falla,
el entorno queda a
medias.

Analogía: cocinar con receta

Tú compras, picas, fríes y emplatas.
Si te equivocas en un paso, el plato sale distinto.

DECLARATIVO: el pedido final

```
plantilla (pseudocódigo)

Resources:
  Red:
    Type: VPC
  Subred:
    Type: Subnet
  Web:
    Type: Instance
    Count: 2
  Balanceador:
    Type: LoadBalancer
```



Motor



El motor calcula los pasos y su
orden.

Analogía: pedir en el restaurante

Tú dices qué quieres en la mesa.
La cocina decide cómo lograrlo.

Fuente: adaptado de AWS CloudFormation User Guide y HashiCorp Terraform Documentation (concepto declarativo)

Cómo piensa un motor declarativo

Compara lo que quieres con lo que existe y ejecuta solo la diferencia.

ESTADO DESEADO

La plantilla dice:



3 instancias web
tipo m5.large

Subred pública y balanceador

ESTADO ACTUAL

Lo que existe en la nube:



2 instancias web
una es m5.xlarge

Subred pública y balanceador: iguales



Compara

PLAN DE CAMBIOS (lo calcula el motor)

+ Crear 1 instancia web
falta la tercera

~ Modificar el tipo de la instancia 2
de m5.xlarge a m5.large

= Red, subred y balanceador
sin cambios

Aplicar el plan
solo se ejecutan los cambios necesarios, en el orden correcto

Fuente: adaptado de HashiCorp Terraform Documentation (plan y apply) y AWS CloudFormation User Guide (conjuntos de cambios)

Idempotencia: repetir sin miedo

Una operación idempotente deja el mismo resultado la primera vez y las diez siguientes.

Script ingenuo: "crear un servidor"

1. **Ejecución** — crea otro
2. **Ejecución** — crea otro
3. **Ejecución** — crea otro

→ **Resultado en la nube**



3 servidores, 3 facturas

cada ejecución agrega uno

Un reintento por falla de red duplica los recursos.

f(f(x)) distinto de f(x)

Declarativo: "debe existir 1 servidor"

1. **Ejecución** — ya existe, sin cambios
2. **Ejecución** — ya existe, sin cambios
3. **Ejecución** — ya existe, sin cambios

→ **Resultado en la nube**



1 servidor, siempre

repetir no cambia el resultado

Un reintento por falla de red no cambia nada: compara y sigue.

f(f(x)) = f(x)

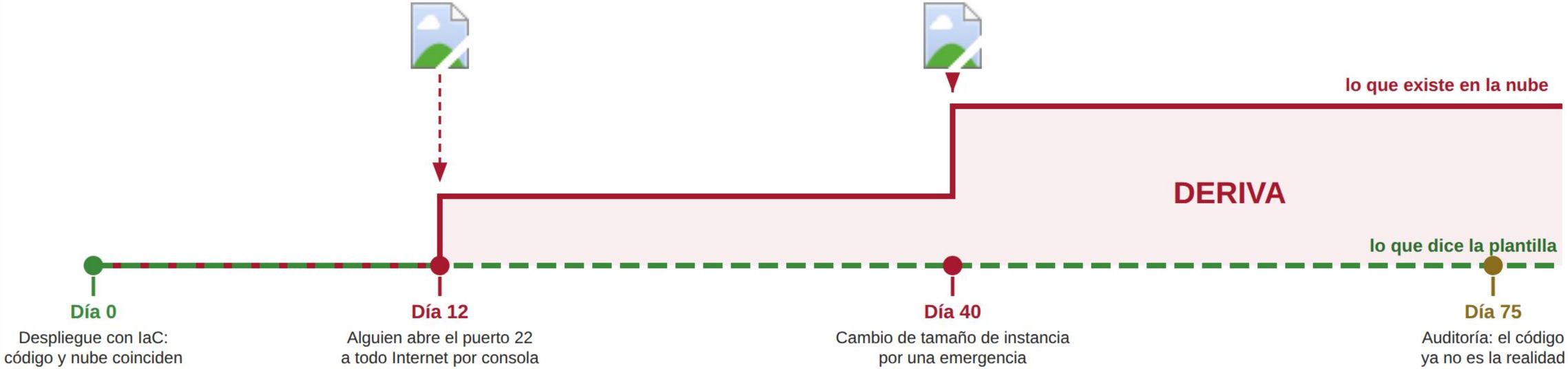
Fuente: adaptado de AWS CloudFormation User Guide y AWS Well-Architected Framework (despliegues repetibles)

DESARROLLO

Deriva de configuración: el código que miente

Cada cambio manual aleja la realidad del código, y la diferencia crece sin que nadie la vea.

Qué dice el código frente a qué hay realmente



Deriva (drift)

Diferencia entre la infraestructura declarada en el código y la que existe de verdad.

Por qué ocurre

Cambios manuales de emergencia, scripts sueltos, otro equipo con permisos amplios.

Por qué duele

La próxima ejecución puede borrar el arreglo manual o fallar al chocar con él.

Fuente: adaptado de AWS CloudFormation User Guide (detección de deriva) y AWS Well-Architected Framework

Infraestructura mutable frente a inmutable

En la inmutable no se arregla un servidor: se reemplaza por uno nuevo construido desde una imagen.

MUTABLE: la mascota que se cura



El mismo servidor se modifica en el sitio, una y otra vez.

Consecuencias

- Nadie sabe qué tiene instalado exactamente.
- No se puede reproducir si se daña.
- Cada servidor termina siendo un "copo de nieve" único.
- Para entenderlo hay que entrar a revisarlo.

INMUTABLE: el ganado que se reemplaza



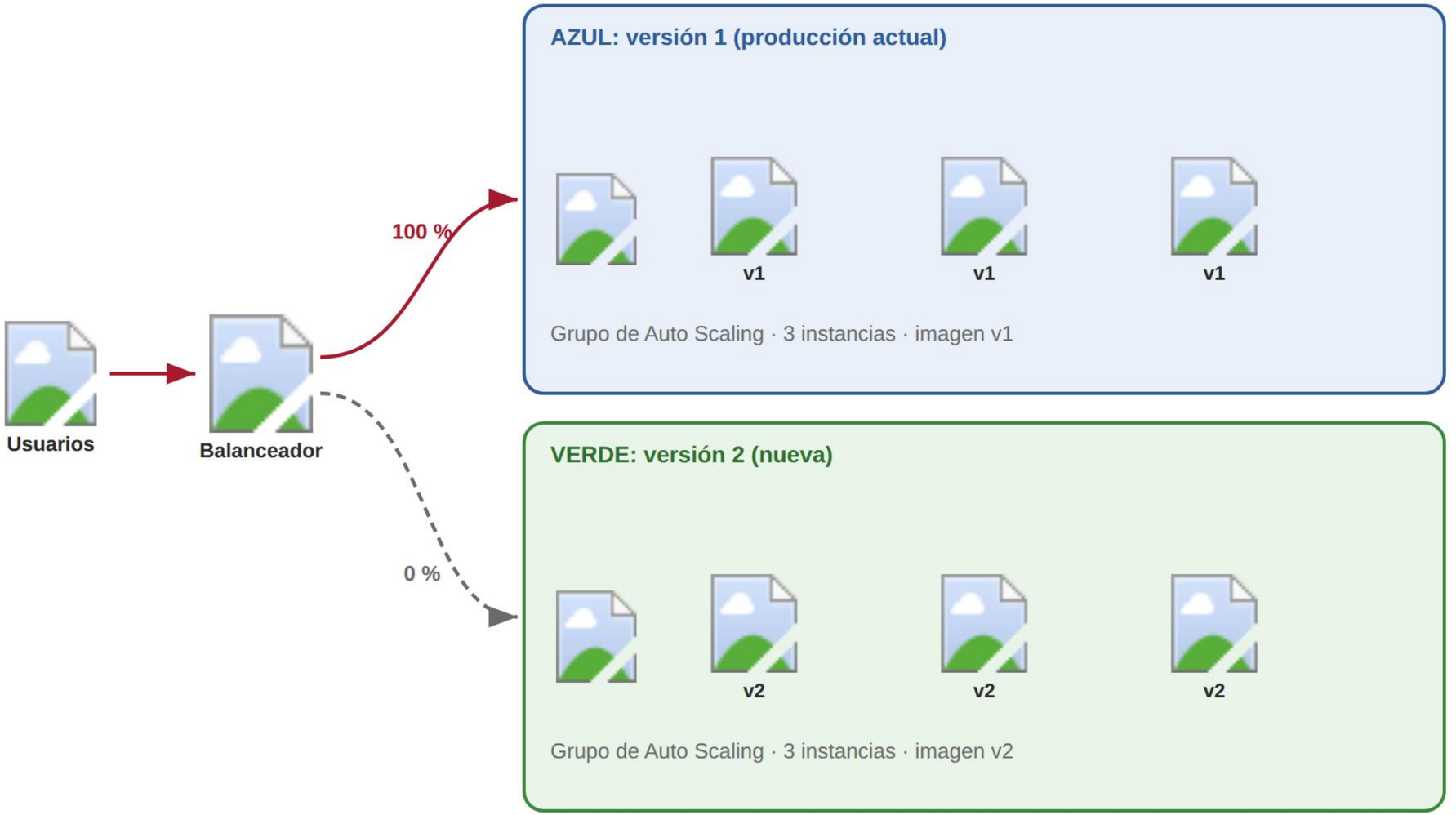
Consecuencias

- Cada servidor sale de una imagen conocida.
- Cambiar es reemplazar, no tocar.
- Si algo falla, se vuelve a la imagen anterior.
- Nadie entra por SSH a "arreglar".

Fuente: adaptado de AWS Well-Architected Framework (despliegues inmutables) y AWS Whitepaper "Overview of Deployment Options"

Reemplazo azul-verde: inmutable en acción

Dos entornos completos conviven; el tráfico cambia de uno al otro y la vuelta atrás es instantánea.



1. **Crear el grupo verde desde la imagen v2**
2. **Pruebas de humo contra el grupo verde**
3. **Mover el tráfico al verde: 100 %**
4. **Conservar el azul unas horas por si hay que volver atrás**

Fuente: adaptado de AWS Whitepaper "Blue/Green Deployments on AWS" y AWS CodeDeploy User Guide

Beneficios que se pueden medir

Automatizar no es una moda: mejora tiempos, fallos y recuperación de forma verificable.

Tiempo para tener un entorno nuevo

Cambios que terminan en falla

Tiempo para recuperarse de una caída

Diferencias entre dev, test y prod

Las cuatro métricas de DORA que mejoran con automatización:

Valores ilustrativos del caso CIC

- Frecuencia de despliegue
- Tiempo de entrega de un cambio
- Tasa de fallos de cambios
- Tiempo de recuperación

Fuente: adaptado de DORA, Accelerate State of DevOps (cuatro métricas); valores ilustrativos del caso Corporación Industrial del Caribe

IaC en los pilares de Well-Architected

Un solo hábito, escribir la infraestructura como código, mejora los seis pilares del marco.

Infraestructura como código



Excelencia operativa

Operaciones como código: todo cambio es versionado y repetible

principio directo



Seguridad

Cambios revisados y auditables; sin accesos manuales



Fiabilidad

Reconstruir un entorno completo en minutos



Eficiencia de rendimiento

Probar otros tamaños con solo cambiar un parámetro



Optimización de costos

Crear y borrar entornos bajo demanda



Sostenibilidad

Solo lo necesario, apagado automático fuera de horario

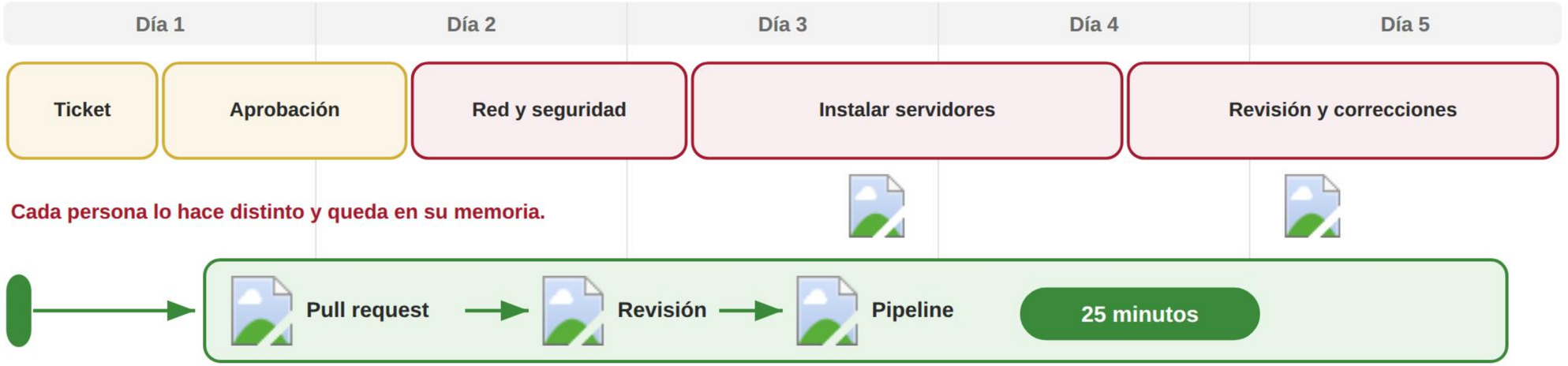
Fuente: adaptado de AWS Well-Architected Framework, seis pilares; principio "Perform operations as code"

CASO APLICADO

Caso CIC: antes y después de automatizar

El mismo pedido de infraestructura pasa de unos 5 días a unos 25 minutos.

Corporación Industrial del Caribe, Barranquilla: necesita un entorno para su nuevo sistema de pedidos de planta.



La plantilla es la documentación viva

Cualquiera la lee, la revisa y la reutiliza para los entornos de pruebas y capacitación. Nada depende de la memoria de una persona.

Lección

El tiempo no se ahorra en el despliegue: se ahorra en no tener que volver a pensarlo cada vez.

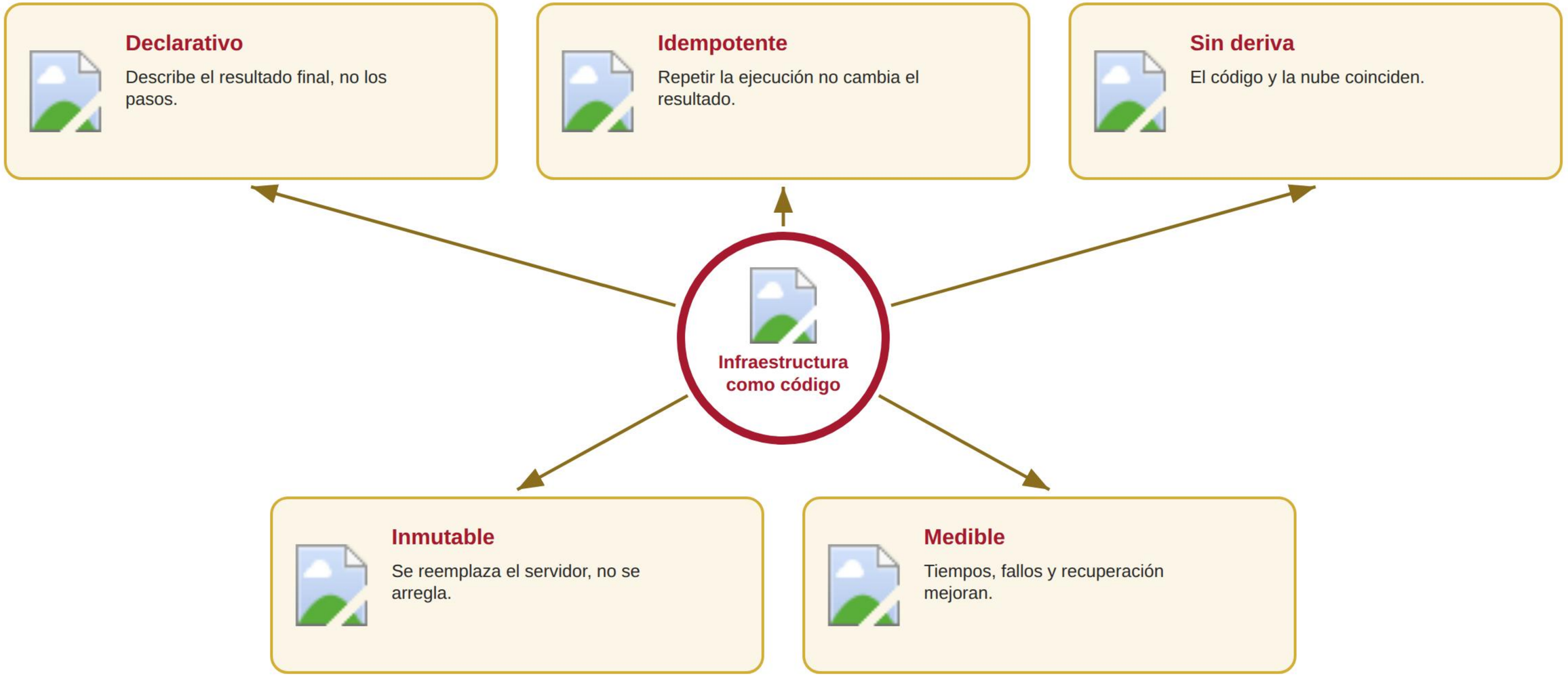
Cifras ilustrativas.

Fuente: elaboración propia (caso Corporación Industrial del Caribe, valores ilustrativos); AWS Well-Architected Framework

CIERRE

Lo que queda de la Parte 1

Cinco ideas para llevar: declarar, repetir sin miedo, evitar la deriva, reemplazar y medir.

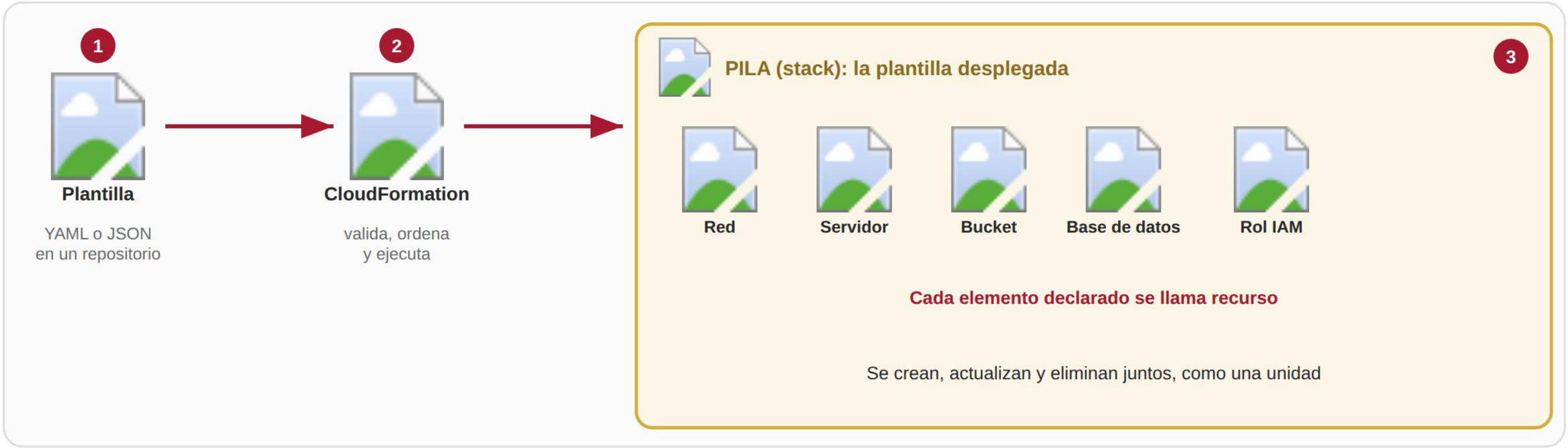


Fuente: síntesis de la Parte 1; AWS Well-Architected Framework y DORA

ENCUADRE

La plantilla es el plano; la pila, la obra

CloudFormation convierte un archivo de texto en recursos reales y los administra como una sola unidad.



Plantilla

El plano: describe qué debe existir.

Pila

Una instancia de ese plano en una cuenta y región.

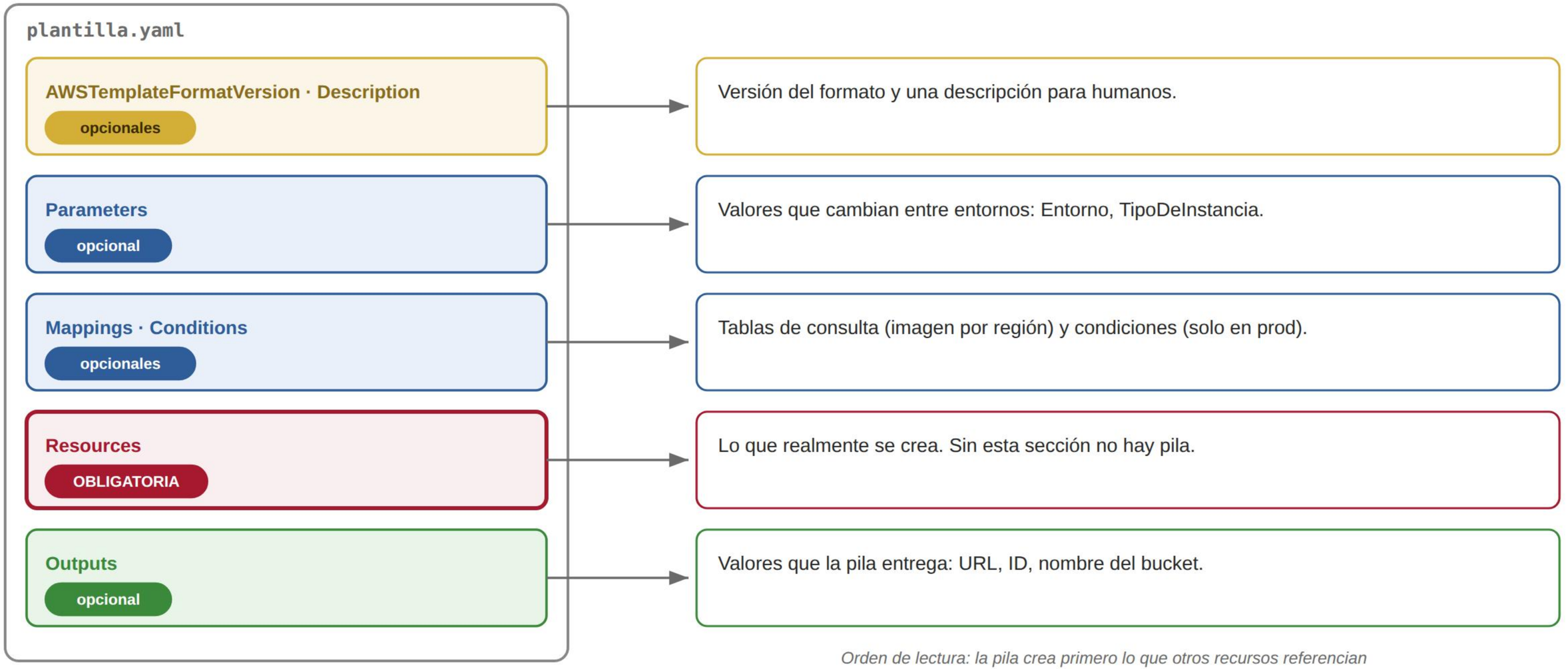
Recurso

Cada pieza concreta: red, servidor, bucket.

Fuente: adaptado de AWS CloudFormation User Guide, "How CloudFormation works"

Anatomía de una plantilla CloudFormation

Solo Resources es obligatoria; el resto hace la plantilla reutilizable y legible.



Orden de lectura: la pila crea primero lo que otros recursos referencian

Fuente: adaptado de AWS CloudFormation User Guide, "Template anatomy"

Una plantilla corta, leída como diagrama

Parámetro de entrada, recurso en el centro y salida: tres bloques que se repiten en toda plantilla.

reportes.yaml

```yaml
AWSTemplateFormatVersion: "2010-09-09"
Parameters:
  Entorno:
    Type: String
    AllowedValues: [dev, test, prod]
Resources:
  Bucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: !Sub cic-reportes-${Entorno}-${AWS::AccountId}
      VersioningConfiguration:
        Status: Enabled
Outputs:
  NombreBucket:
    Value: !Ref Bucket
```

PARÁMETRO: Entorno

Al crear la pila eliges dev, test o prod. El mismo archivo sirve para los tres entornos.

RECURSO: Bucket



Un bucket de S3 con versionado activado. El nombre se arma con !Sub: entorno + número de cuenta.

SALIDA: NombreBucket

!Ref Bucket devuelve el nombre real; otras pilas pueden importarlo.

Al crear la pila con Entorno = dev



`cic-reportes-dev-111122223333`

versionado: Enabled · cuenta de ejemplo

Quince líneas bastan para crear un recurso real, repetible y revisable.

Fuente: adaptado de AWS CloudFormation User Guide (AWS::S3::Bucket, funciones intrínsecas Sub y Ref); nombre de cuenta ilustrativo

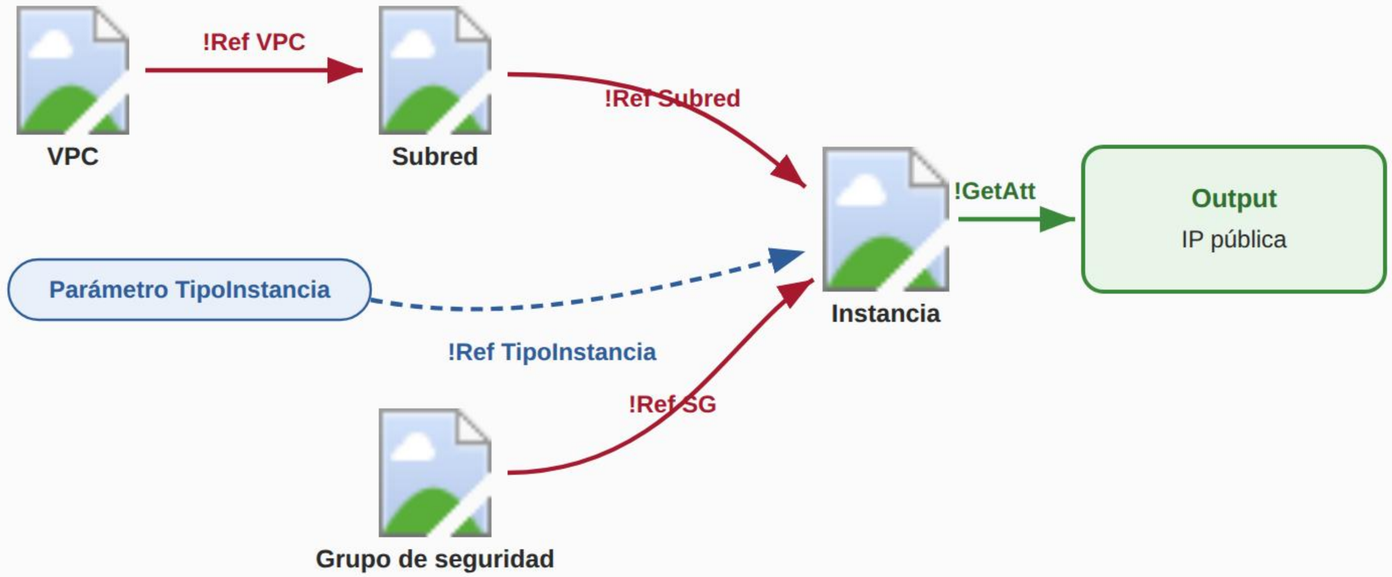
Parámetros y funciones intrínsecas

Con !Ref y !GetAtt un recurso apunta a otro, y CloudFormation deduce el orden de creación.

Funciones intrínsecas

!Ref	Devuelve el valor de un parámetro o el ID de un recurso
!Sub	Sustituye variables dentro de un texto
!GetAtt	Lee un atributo del recurso, como su IP o su ARN
!FindInMap	Busca un valor en una tabla de Mappings
!If	Elige un valor según una condición

Dependencias: CloudFormation las deduce de las referencias



Orden de creación resultante

VPC › Subred y SG › Instancia › Output

Fuente: adaptado de AWS CloudFormation User Guide, "Intrinsic function reference"

SEMANA 11 · AUTOMATIZACIÓN DE INFRAESTRUCTURA (IAC) — REPASO U2

Plantillas, herramientas y entrega continua

Parte 2 de 3 · Unidad 2

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 11 · Periodo 2026-2 · Barranquilla, Colombia

Ciclo de vida de la pila, Terraform y Bicep, y el pipeline de entrega continua

DESARROLLO

Ciclo de vida de una pila

Cada operación termina en un estado final claro; un fallo no deja recursos a medias.

CREAR

- CREATE_IN_PROGRESS → CREATE_COMPLETE — Todo bien: la pila queda lista para usarse.
- CREATE_IN_PROGRESS → (falla) → ROLLBACK_IN_PROGRESS → ROLLBACK_COMPLETE — Falla de un recurso: se deshace todo. La pila solo se puede eliminar y volver a crear.

ACTUALIZAR

- UPDATE_IN_PROGRESS → UPDATE_COMPLETE — Cambio aplicado con éxito.
- UPDATE_IN_PROGRESS → (falla) → UPDATE_ROLLBACK_IN_PROGRESS → UPDATE_ROLLBACK_COMPLETE — Falla al actualizar: vuelve sola a la versión anterior y sigue funcionando.

ELIMINAR

- DELETE_IN_PROGRESS → DELETE_COMPLETE — Se borran los recursos, salvo los marcados con DeletionPolicy: Retain.



Dónde mirar cuando algo falla

Consola de CloudFormation, pestaña Eventos: el primer evento FAILED indica la causa raíz.

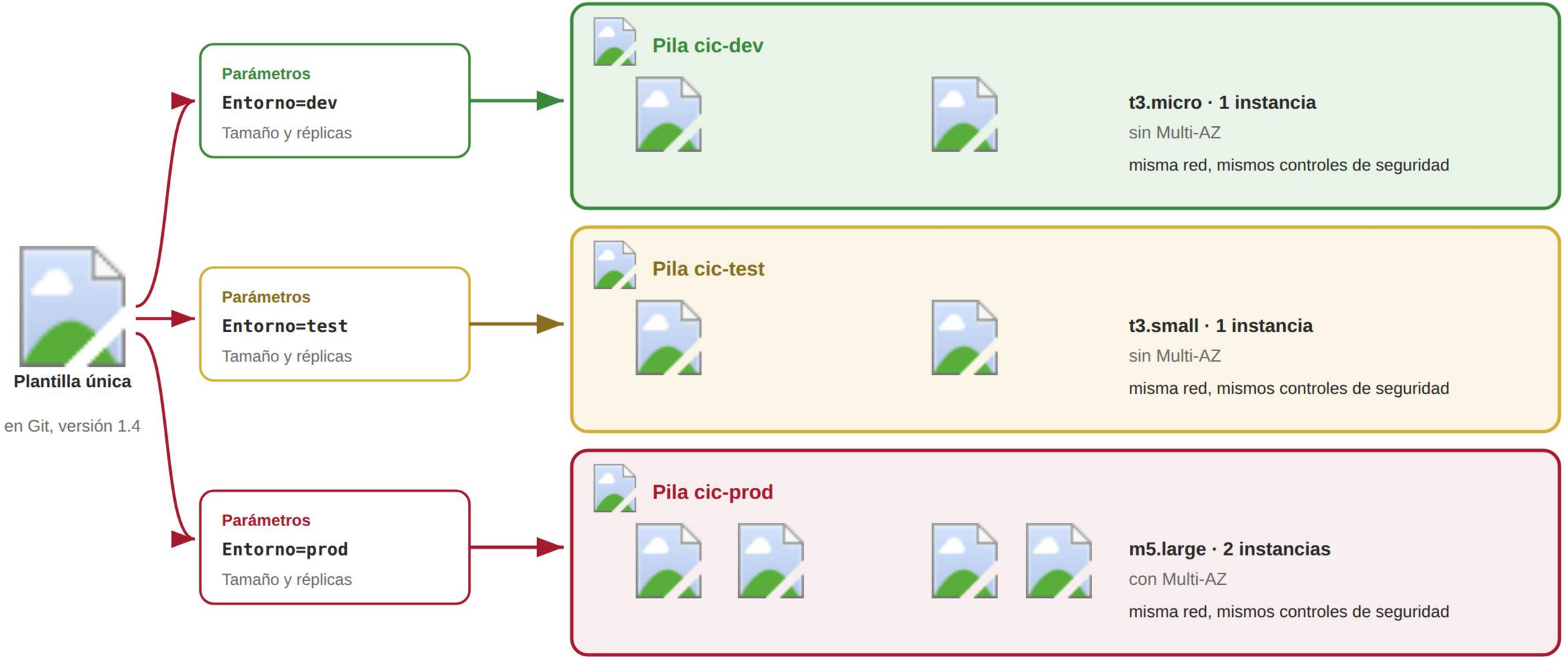
Los eventos posteriores suelen ser consecuencia, no causa.

Fuente: adaptado de AWS CloudFormation User Guide, "Stack status codes"

DESARROLLO

Un mismo plano, tres entornos

Los parámetros cambian el tamaño; la forma y los controles son idénticos en dev, test y prod.



Fuente: adaptado de AWS Well-Architected Framework (entornos reproducibles); caso Corporación Industrial del Caribe

DESARROLLO

Conjuntos de cambios: ver antes de aplicar

Un conjunto de cambios muestra qué se agrega, modifica o reemplaza sin tocar la pila.

1 Cambias la plantilla



versión 2 en Git

2 Creas el conjunto de cambios



no se toca nada

3 Revisas la vista previa



qué se agrega, cambia o borra

4 Ejecutas o descartas



recién ahora se aplica

Vista previa del conjunto de cambios (ejemplo)

Acción	ID lógico	Tipo de recurso	Reemplazo	
Modify	**Instancia**	`AWS::EC2::Instance`	True	Cambió ImageId
Modify	**GrupoSeguridad**	`AWS::EC2::SecurityGroup`	False	Regla modificada
Add	**AlarmaCPU**	`AWS::CloudWatch::Alarm`	—	Recurso nuevo
Remove	**BucketTemporal**	`AWS::S3::Bucket`	—	Ya no está



Atención al reemplazo

Replacement True: se destruye el recurso y se crea otro nuevo. Si guardaba datos, se pierden.

Por qué importa

Es el equivalente a un "plan": ves el efecto antes de pagar sus consecuencias.

Fuente: adaptado de AWS CloudFormation User Guide, "Updating stacks using change sets"

Detección de deriva en CloudFormation

Compara lo que dice la plantilla con lo que existe y marca los recursos que cambiaron por fuera.



Pila → **Detectar deriva** →

IN_SYNC

Los recursos son iguales a la plantilla.

DRIFTED

Al menos un recurso cambió fuera de CloudFormation.

Estado por recurso

Bucket	IN_SYNC
Instancia	IN_SYNC
GrupoSeguridad	MODIFIED
BucketTemporal	DELETED

Detalle de GrupoSeguridad (diferencias de propiedades)

```
PropertyPath: /SecurityGroupIngress/0/CidrIp
ExpectedValue: 10.20.0.0/16        # plantilla
ActualValue:   0.0.0.0/0           # realidad
DifferenceType: NOT_EQUAL
```

Alguien abrió el puerto a todo Internet por la consola. El código decía solo la red de la oficina (CIDR de ejemplo).



CloudFormation detecta la deriva, pero no la corrige sola

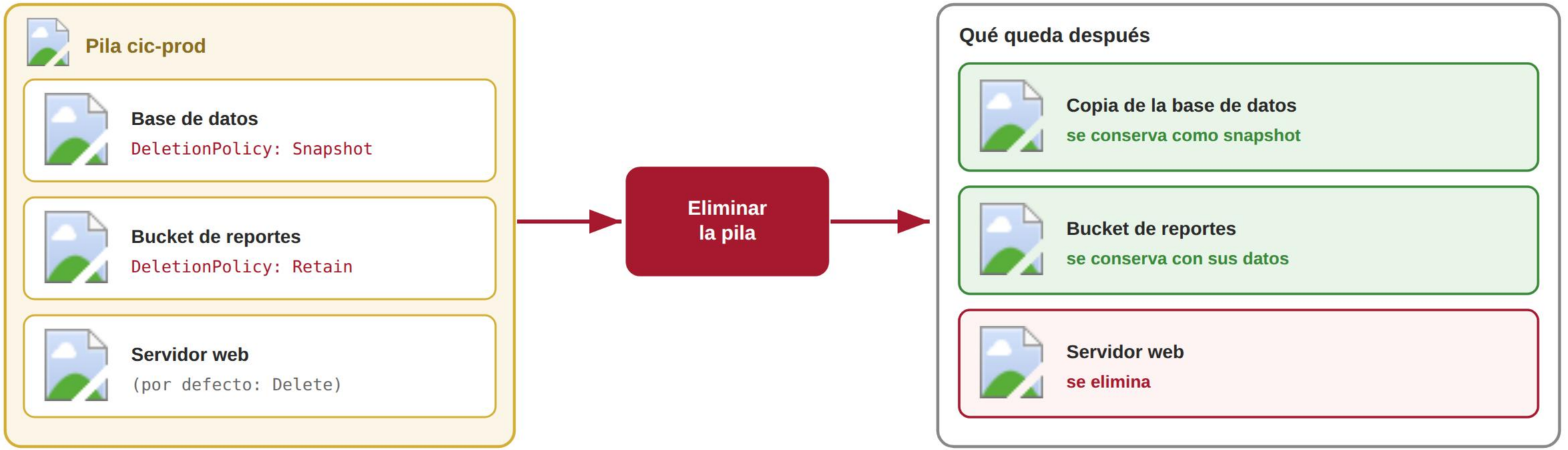
Tú decides: revertir el cambio manual, o actualizar la plantilla para que refleje la nueva realidad.

Tampoco ve los recursos que se crearon fuera de la pila.

Fuente: adaptado de AWS CloudFormation User Guide, "Detect unmanaged configuration changes (drift)"

Proteger lo que no se puede perder

DeletionPolicy y protecciones de pila evitan que borrar un entorno destruya datos valiosos.



Tres capas de protección contra borrados accidentales

DeletionPolicy
Retain o Snapshot por recurso

Protección contra terminación
impide eliminar la pila entera

Política de pila
impide actualizar recursos críticos

Fuente: adaptado de AWS CloudFormation User Guide (DeletionPolicy, termination protection y stack policies)

Terraform: init, plan y apply

Un solo flujo y un lenguaje, HCL, para administrar recursos en muchas nubes.

```
main.tf (HCL)
resource "aws_s3_bucket" "reportes" {
  bucket = "cic-reportes-${var.entorno}"
}

variable "entorno" {
  type = string
}
```



Terraform CLI

Lee los archivos .tf y habla con las APIs de cada nube a través de proveedores.

1 — terraform init

Descarga proveedores y configura el estado

2 — terraform plan

Calcula qué cambiaría y lo muestra

3 — terraform apply

Ejecuta el plan tras tu confirmación

4 — terraform destroy

Elimina lo que Terraform creó

Un mismo flujo, muchos proveedores


AWS


Azure


Google Cloud


OpenStack


Kubernetes



El plan es la red de seguridad

Muestra con símbolos + (crear), ~ (modificar) y - (destruir) qué ocurrirá, antes de tocar nada.

Fuente: adaptado de HashiCorp Terraform Documentation, "Core workflow"

El estado de Terraform

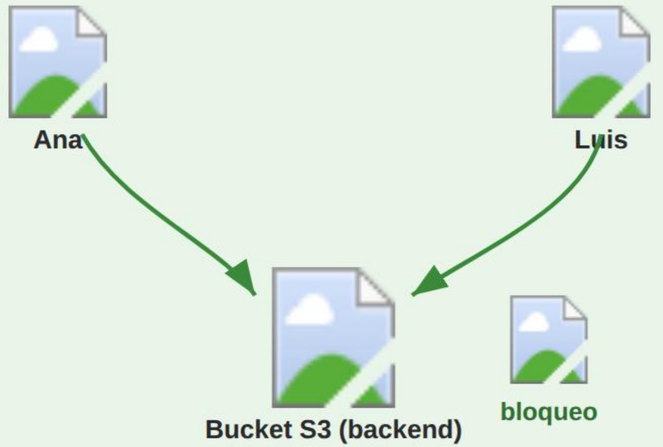
Terraform recuerda lo que creó en un archivo de estado; compartirlo bien es parte del diseño.

Estado local: cada quien con su copia



Dos copias distintas: Terraform duplica o destruye recursos.

Estado remoto: una sola fuente de verdad



Con versionado, cifrado y bloqueo: nadie aplica al mismo tiempo.

¿Qué guarda el estado?

aws_s3_bucket.reportes → **bucket real: cic-reportes-dev**



El mapa entre cada recurso del código y su objeto real. Puede contener datos sensibles: cifra el almacenamiento y limita el acceso.

Fuente: adaptado de HashiCorp Terraform Documentation, "State" y "Backend: s3"

Bicep: el lenguaje declarativo de Azure

La misma idea, con otra sintaxis: describir recursos y dejar que Azure Resource Manager los cree.

almacenamiento.bicep

```
param entorno string = 'dev'

resource sa 'Microsoft.Storage/storageAccounts@2023-05-01' = {
  name: 'cicrep${entorno}${uniqueString(resourceGroup().id)}'
  location: resourceGroup().location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
}

output nombre string = sa.name
```

Qué ves en este archivo

parámetro · recurso · salida

Más corto y legible que JSON. El tipo y la versión de API van en la declaración del recurso.

Bicep es un lenguaje de dominio para Azure: es declarativo y sin estado propio.

Compila a una plantilla ARM y la ejecuta Azure Resource Manager



Azure Resource Manager

Comandos clave (Azure CLI)

`az deployment group what-if`

Vista previa de cambios, como el plan

`az deployment group create`

Despliega el archivo .bicep en un grupo de recursos

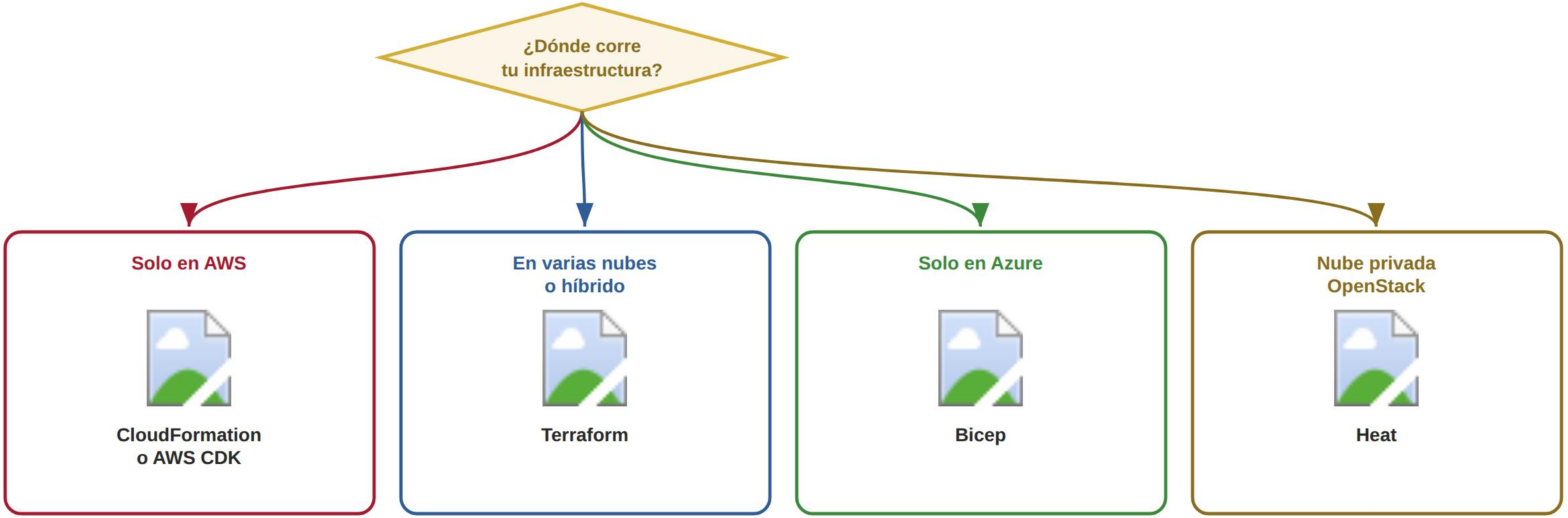
`az bicep build`

Compila a JSON de ARM

Fuente: adaptado de Microsoft Learn, "Bicep documentation" (CC BY 4.0)

¿Cuál elegir? Depende de dónde corres

Las herramientas comparten la idea; cambian el lenguaje, el alcance y dónde guardan el estado.





AWS CDK: escribes en Python o TypeScript y genera plantillas de CloudFormation

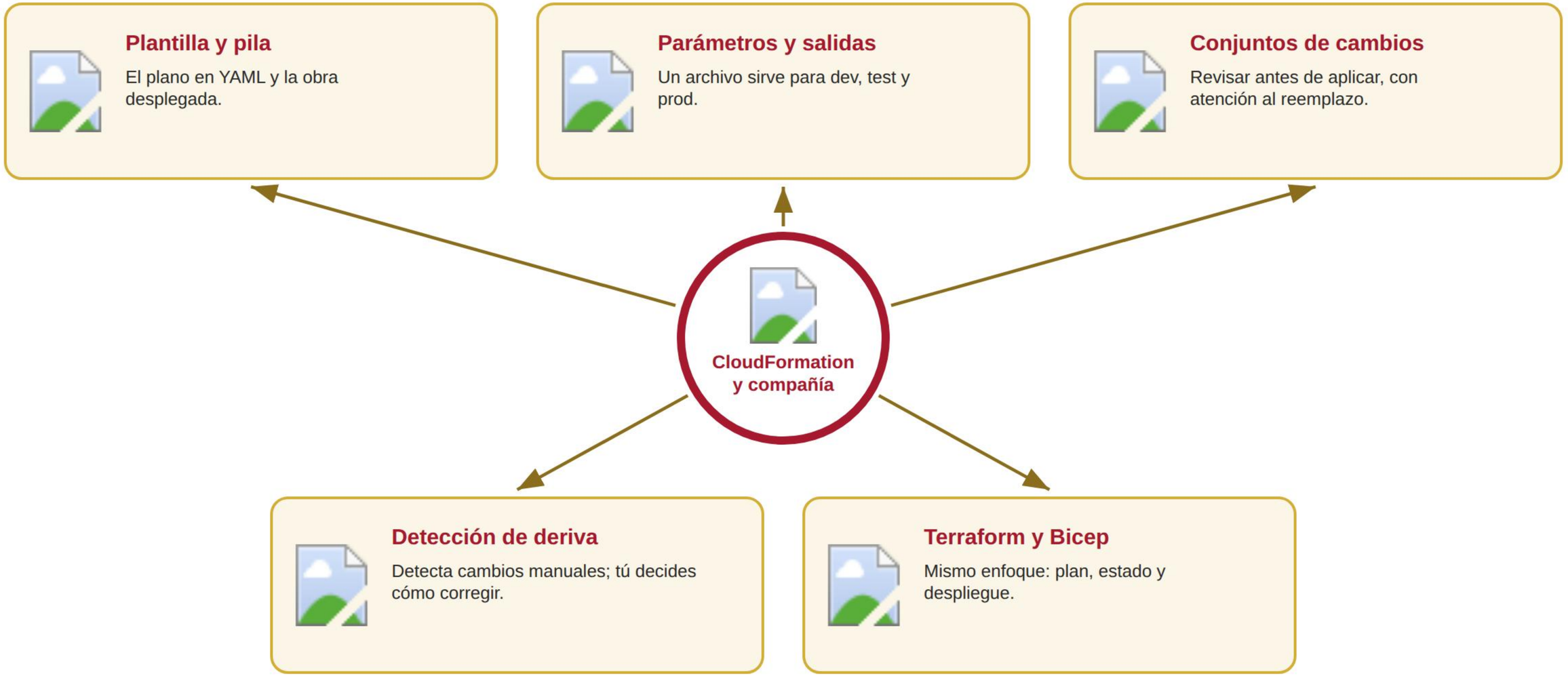
El motor sigue siendo declarativo; solo cambia cómo se escribe la plantilla.

Fuente: elaboración propia a partir de AWS CloudFormation, HashiCorp Terraform, Microsoft Learn (Bicep) y OpenStack Heat Documentation

CIERRE

Lo que queda de la Parte 2

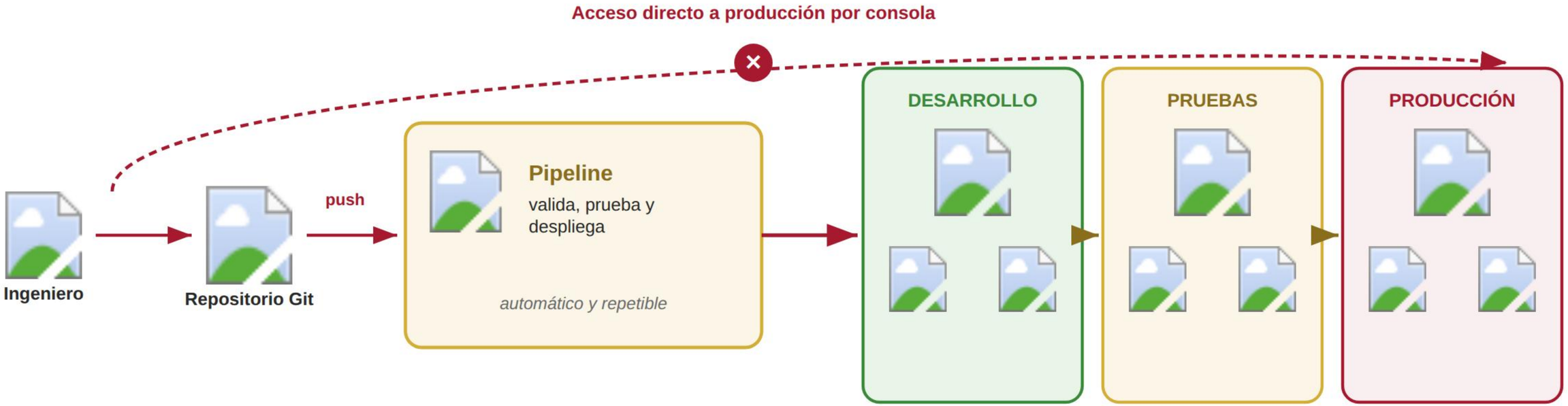
CloudFormation, Terraform y Bicep aplican el mismo ciclo: declarar, revisar, aplicar y vigilar la deriva.



Fuente: síntesis de la Parte 2; AWS CloudFormation User Guide, HashiCorp Terraform Documentation, Microsoft Learn

La plantilla también es software

Si la infraestructura es código, se versiona, se revisa, se prueba y se entrega como cualquier programa.



Antes

Cada persona entra a producción y cambia lo que cree necesario. No hay historial ni revisión.

Entrega continua

Todo cambio pasa por Git y por el pipeline. La consola queda para mirar, no para cambiar.

Resultado

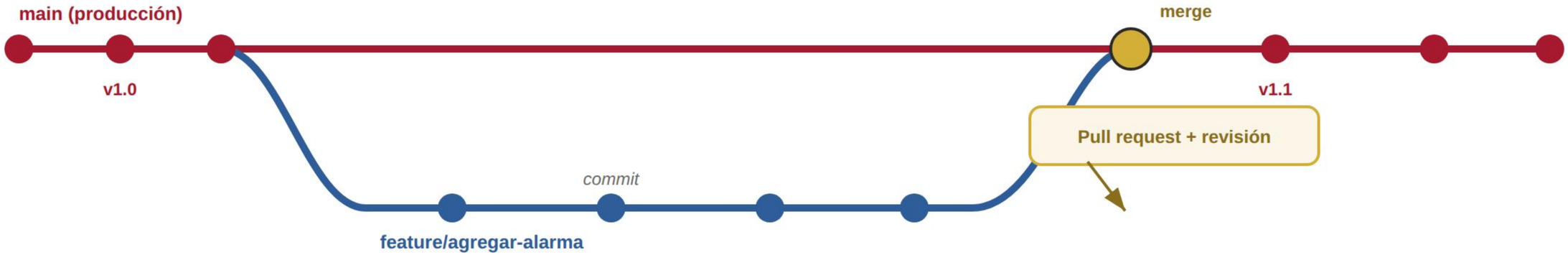
Cambios pequeños, frecuentes, revisados y reversibles.

Fuente: adaptado de AWS Well-Architected Framework (pilar de excelencia operativa) y DORA, Accelerate State of DevOps

Control de versiones: la base de todo

Cada cambio de infraestructura es un commit en Git, trabajado en una rama y unido por pull request.

Historial de Git: ramas y pull request



Commit

Punto de restauración con autor, fecha y motivo.

Rama y pull request

Se trabaja aparte y se une solo tras revisión.

Historial = auditoría

Quién cambió qué, cuándo y por qué, para siempre.

Fuente: adaptado de Pro Git (git-scm.com, CC BY-NC-SA 3.0) y AWS Prescriptive Guidance, "CI/CD for infrastructure as code"

Qué va (y qué no) en el repositorio

El repositorio guarda el plano completo, pero nunca los secretos: esos se referencian, no se copian.

```
cic-infraestructura/
├─ plantillas/
│  ├─ red.yaml
│  ├─ datos.yaml
│  └─ computo.yaml
├─ parametros/
│  ├─ dev.json
│  ├─ test.json
│  └─ prod.json
├─ pipelines/
│  └─ buildspec.yml
├─ pruebas/
└─ README.md
```

Un repositorio por sistema, con plantillas, parámetros por entorno, pipeline y pruebas.

Sí va en el repositorio

Plantillas, parámetros no sensibles, scripts del pipeline, pruebas y documentación.



Nunca va en el repositorio

Contraseñas, claves de acceso, tokens y certificados privados. Quedan en el historial para siempre.





Secrets Manager



Parameter Store

La plantilla solo guarda una referencia al secreto.

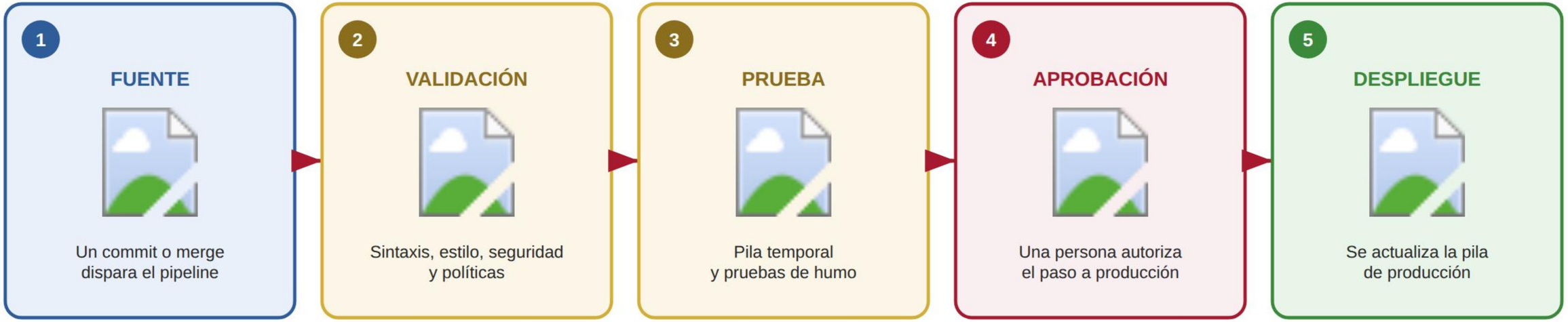
```
MasterUserPassword: '{{resolve:secretsmanager:cic/db:SecretString:password}}'
```

referencia dinámica: el valor se resuelve al desplegar

Fuente: adaptado de AWS CloudFormation User Guide, "Dynamic references" y AWS Prescriptive Guidance (repositorios de IaC)

El pipeline: fuente, validación y despliegue

Un pipeline convierte un commit en un despliegue pasando por etapas que fallan rápido y con control.

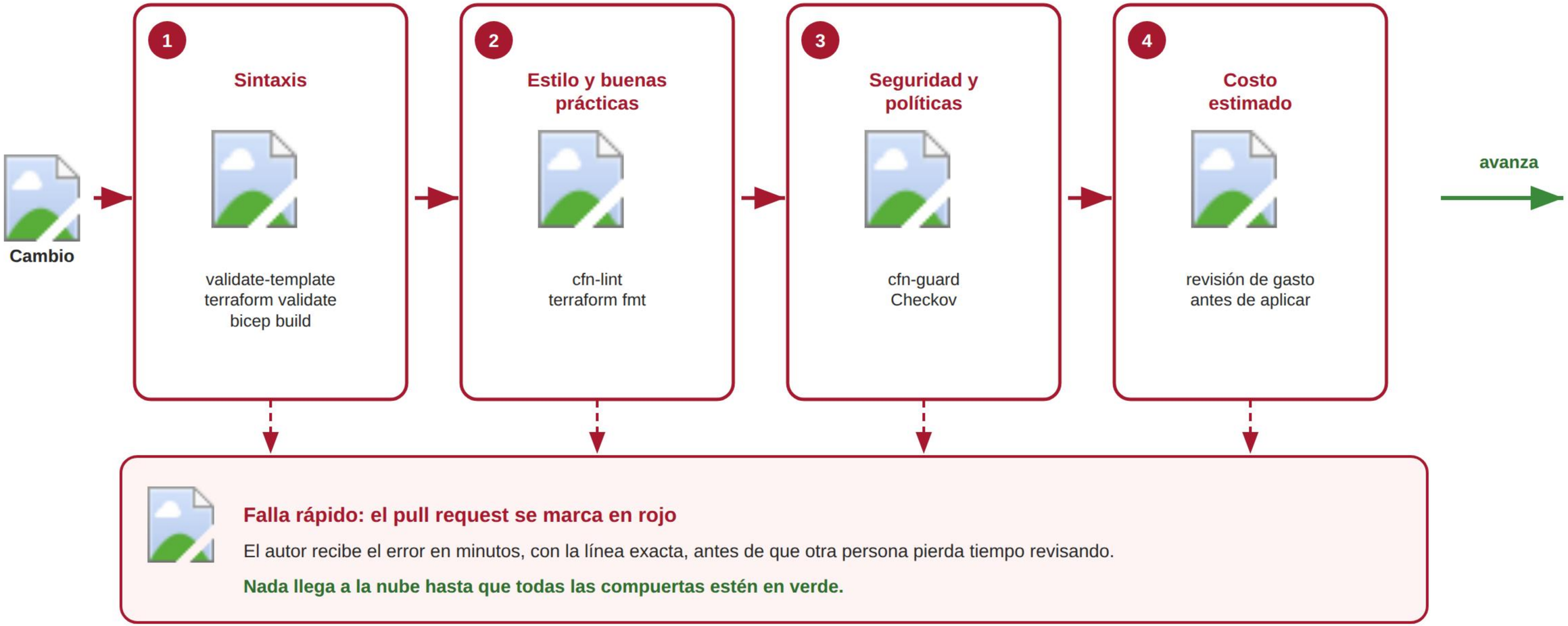


La mayoría de las empresas usa entrega continua en producción de infraestructura crítica.

Fuente: adaptado de AWS Whitepaper "Practicing Continuous Integration and Continuous Delivery on AWS"

Etapa de validación: cuatro compuertas

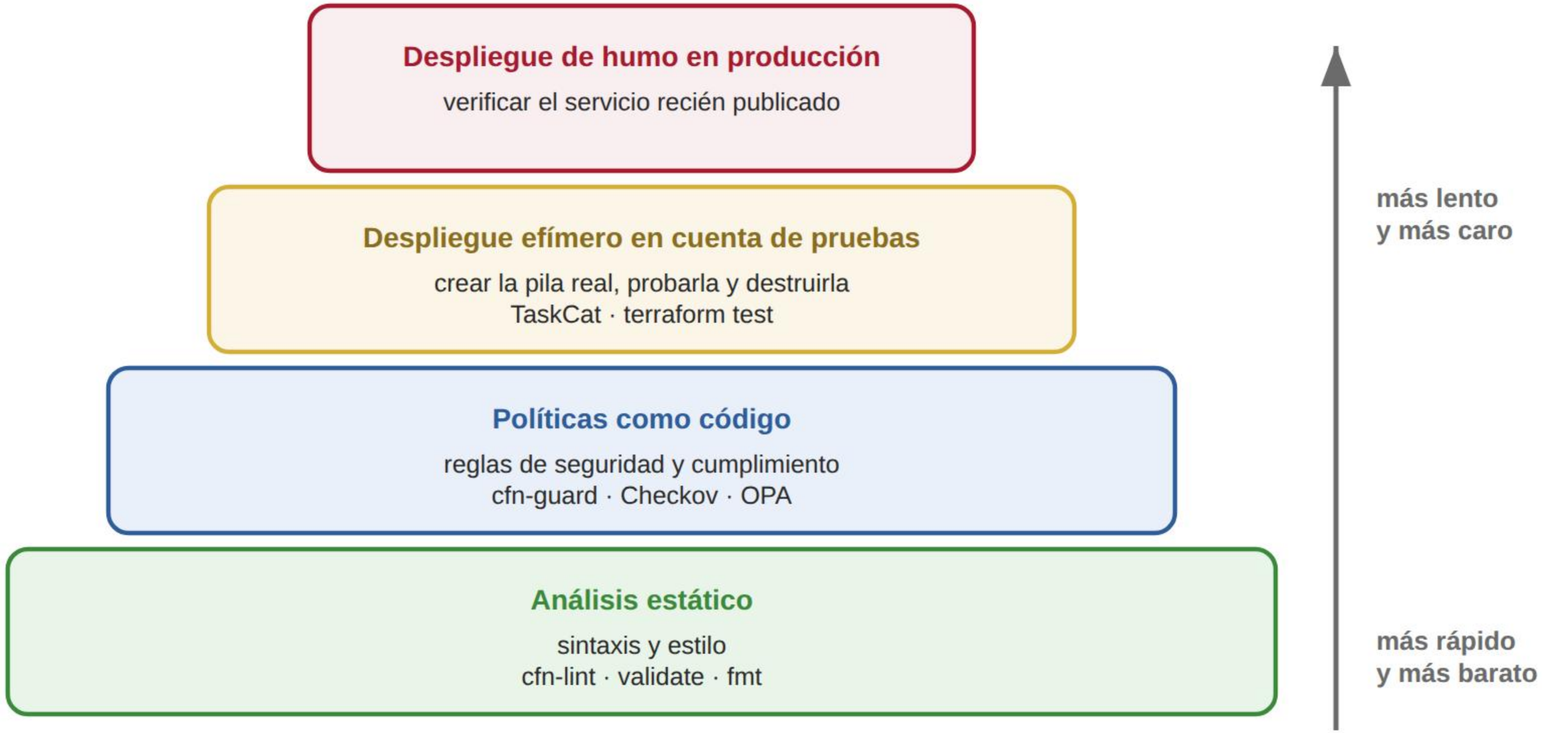
Cada cambio atraviesa revisiones automáticas baratas antes de gastar un solo centavo en la nube.



Fuente: adaptado de AWS CloudFormation Guard, cfn-lint y HashiCorp Terraform Documentation (validate, fmt)

Pruebas de plantillas: la pirámide

Muchas pruebas estáticas rápidas, algunas de políticas y pocas de despliegue real.



Cómo se prueba una plantilla

No se prueba "a ojo": se despliega en una cuenta aislada.

Se comprueba que el servicio responda y que las salidas existan.

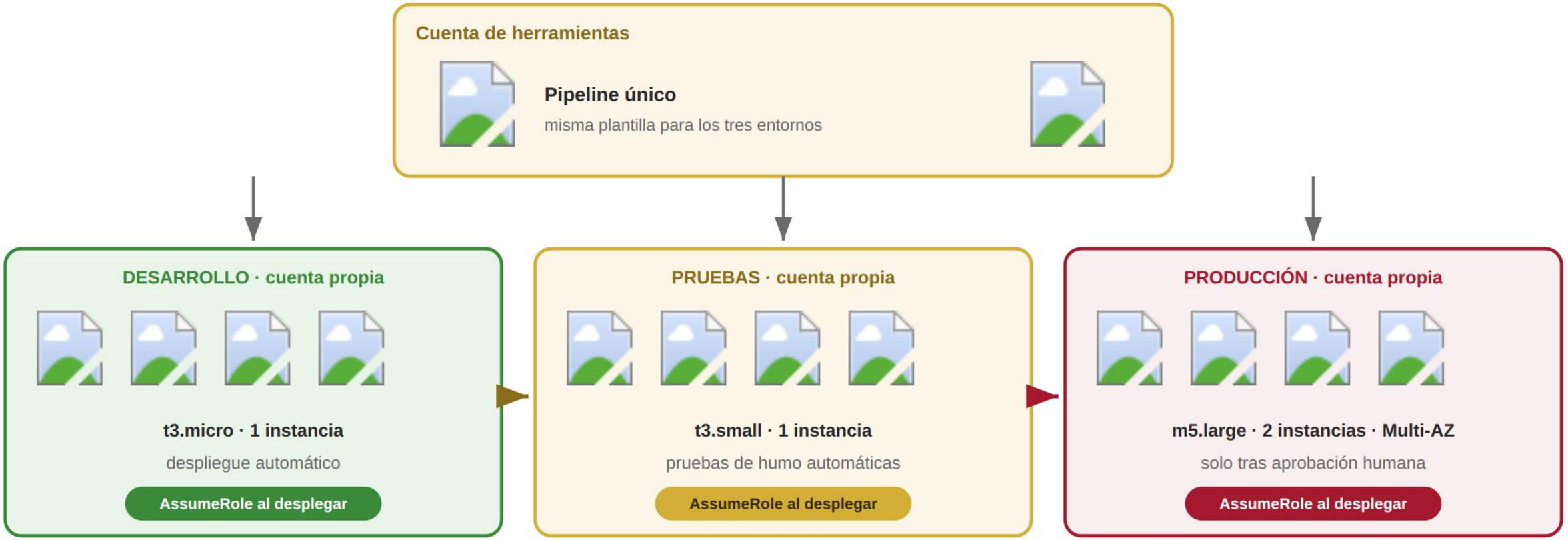
Se elimina la pila al final: no queda gasto residual.

Regla práctica: muchas pruebas rápidas en la base y pocas pruebas lentas en la cima.

Fuente: adaptado de AWS Prescriptive Guidance (pruebas de IaC), AWS TaskCat y HashiCorp Terraform Documentation (terraform test)

Entornos: dev, test y prod en cuentas separadas

El mismo pipeline promueve la misma plantilla de un entorno al siguiente, con parámetros distintos.



Cuentas separadas dentro de AWS Organizations: un error en desarrollo no puede tocar producción.

El pipeline asume un rol distinto en cada cuenta; la aprobación manual protege el último paso.

Fuente: adaptado de AWS Well-Architected Framework y AWS Prescriptive Guidance (estrategia multi-cuenta con AWS Organizations)

Revisión por pares: el segundo par de ojos

Un cambio de infraestructura se aprueba como un cambio de código: con diff, comentarios y comprobaciones.



Pull request #42: agregar acceso SSH para soporte

```
infra/plantillas/computo.yaml
  SecurityGroupIngress:
    - IpProtocol: tcp
      FromPort: 22
      ToPort: 22
-     CidrIp: 0.0.0.0/0
+     CidrIp: 10.20.0.0/16
```



María (revisora)

Abrir el puerto 22 a todo Internet es riesgoso. Limitémoslo a la red de la oficina.

Autor: corregido en el commit siguiente.

Comprobaciones automáticas

- ✓ **Sintaxis y cfn-lint**
- ✓ **Política: sin SSH abierto al mundo**
- ✓ **Conjunto de cambios publicado**
- … **1 aprobación de un revisor**



Reglas de la rama main

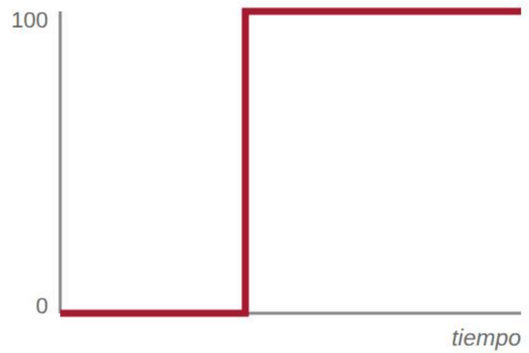
Protegida: nadie sube directo, mínimo una revisión humana y todas las comprobaciones en verde.

Fuente: adaptado de GitHub Docs (protected branches, pull requests) y AWS Well-Architected Framework (revisión de cambios)

Estrategias de despliegue

Todo de una vez es rápido y arriesgado; gradual, azul-verde y canario reducen el daño si algo falla.

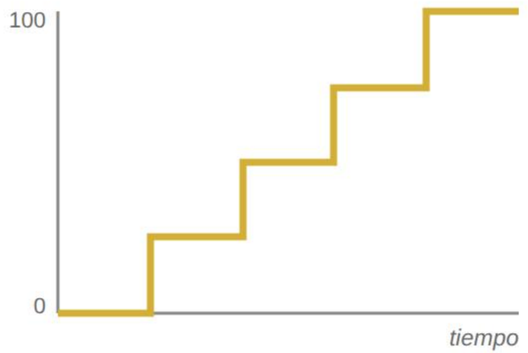
Todo de una vez



Riesgo: alto

Vuelta atrás: volver a desplegar la versión anterior.

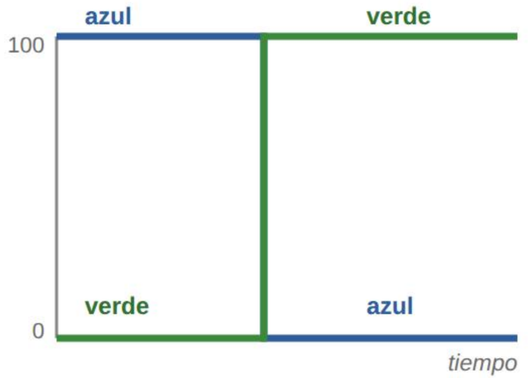
Gradual (rolling)



Riesgo: medio

Se actualiza por lotes; la vuelta atrás es lenta.

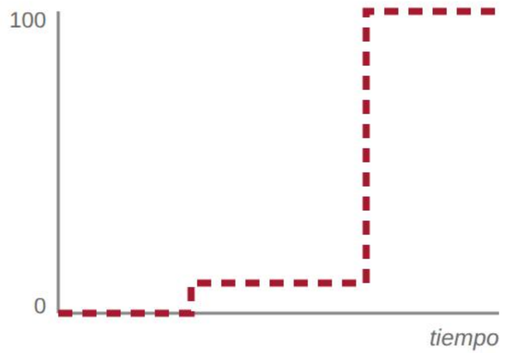
Azul-verde



Riesgo: bajo

Dos entornos completos; volver es cambiar el balanceador.

Canario



Riesgo: bajo

Un 10 % prueba primero; si hay errores, se detiene.

Cuál usar depende del riesgo y del costo

CodeDeploy, ECS y Lambda ofrecen estas estrategias de forma nativa. La infraestructura base (red, datos) se cambia con conjuntos de cambios; las estrategias de arriba se aplican sobre todo a la capa de aplicación.

Más control = más costo temporal

Fuente: adaptado de AWS Whitepaper "Overview of Deployment Options on AWS" y AWS CodeDeploy User Guide

SEMANA 11 · AUTOMATIZACIÓN DE INFRAESTRUCTURA (IAC) — REPASO U2

Entrega segura, repaso U2 y laboratorio

Parte 3 de 3 · Unidad 2

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

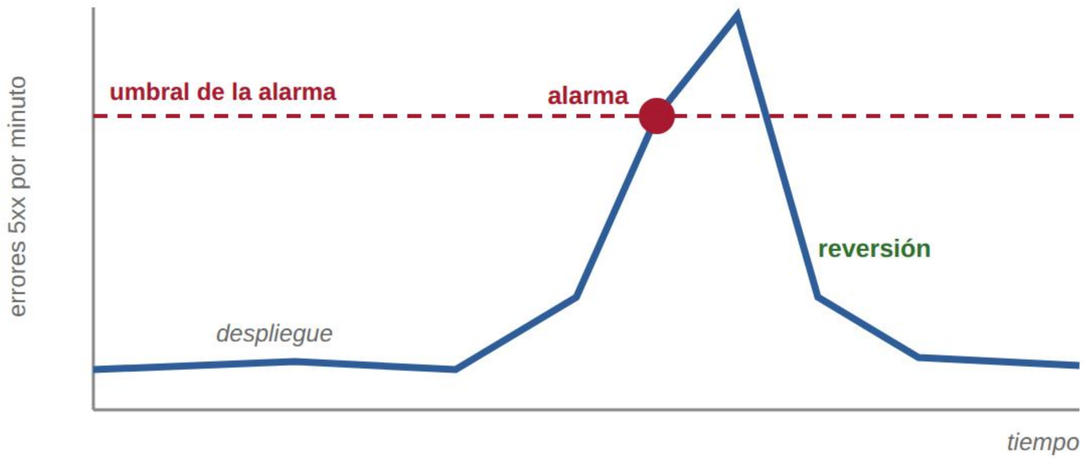
Semana 11 · Periodo 2026-2 · Barranquilla, Colombia

Reversión, GitOps, repaso de la Unidad 2 y el laboratorio de IaC

Reversión: volver atrás con confianza

Una alarma puede deshacer un mal despliegue sola; si no, volver a la versión anterior es un commit.

Alarma que dispara la reversión



Reversión automática

1. Alarma de CloudWatch se activa
2. CloudFormation detecta la alarma y cancela la actualización
3. Los recursos vuelven a la versión anterior

Se configura con alarmas de rollback en la pila o con reversión automática en CodeDeploy.

Sin intervención humana a las 3 de la mañana.

Reversión manual por Git: volver es un commit más

1. **git revert del cambio malo**
2. **El pipeline corre de nuevo**
3. **Se despliega la versión anterior**
4. **Queda registrado en el historial**

Fuente: adaptado de AWS CloudFormation User Guide (rollback triggers) y AWS CodeDeploy User Guide (automatic rollback)

GitOps: Git como fuente de verdad

Un controlador compara continuamente Git con la realidad y corrige cualquier diferencia.

Fuente de verdad



Repositorio Git

Aquí vive el estado deseado, versionado y revisado por pull request.



1 · merge del PR

2

Controlador en el clúster



Argo CD



Flux

Un agente que observa el repositorio y el clúster todo el tiempo.

compara deseado frente a real

3

Clúster o nube (estado real)



Nodo



Pod



VPC

Lo que realmente está en ejecución.

si alguien cambia algo a mano,
el controlador lo revierte

deriva detectada: autocorrección

Modelo "push" (pipeline clásico)

El pipeline empuja el cambio a la nube con credenciales propias. Hay que protegerlas.

Modelo "pull" (GitOps)

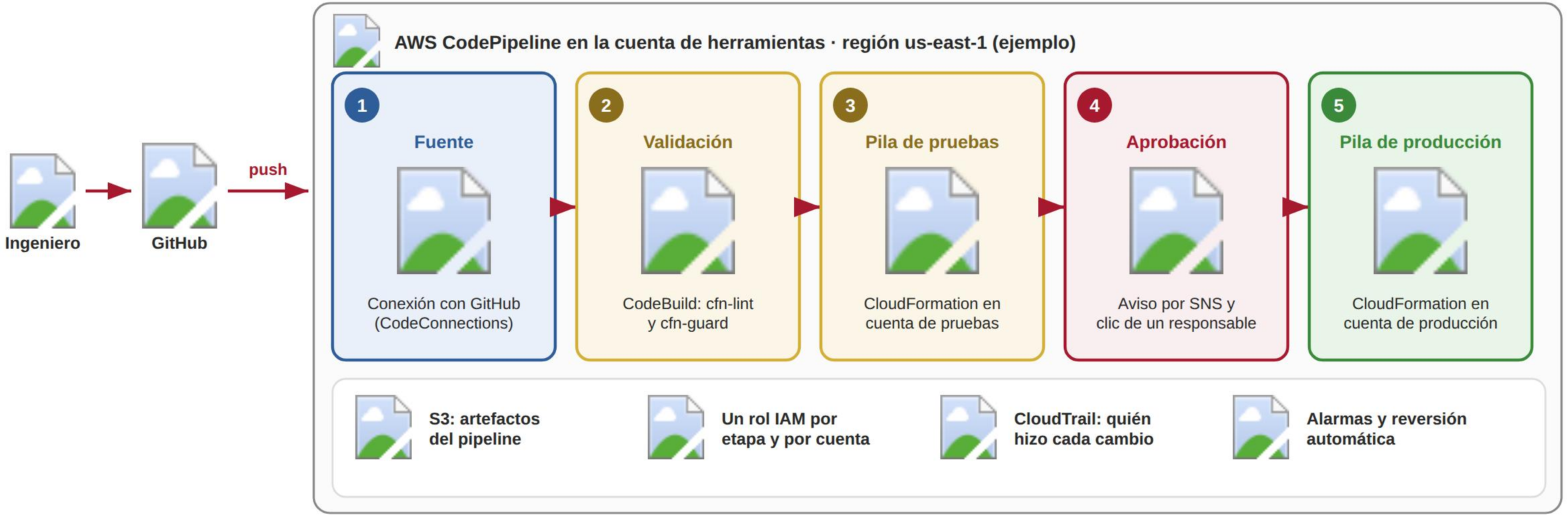
Un agente dentro del entorno jala el estado de Git. La nube no expone credenciales hacia fuera.

Fuente: adaptado de OpenGitOps (principios, CNCF), Argo CD Documentation y Flux Documentation

CASO APLICADO

Caso CIC: el pipeline completo en AWS

Del push en GitHub a la pila de producción: validación, pruebas, aprobación y despliegue sin consola.



Del commit a producción

unos 25 minutos más la aprobación

Cambios por semana

antes 2, ahora varios al día (ilustrativo)

Revisión

100 % de los cambios pasan por pull request

Fuente: adaptado de AWS CodePipeline User Guide; caso CIC (valores ilustrativos)

Seguridad de la cadena de entrega

El pipeline también es un blanco: se protege con credenciales temporales y roles de mínimo privilegio.

Antes: claves de acceso largas



guardadas como secreto del repositorio

Se filtran, no caducan, sirven desde cualquier lugar.

Ahora: federación OIDC

Sin claves guardadas. Cada ejecución recibe credenciales temporales de pocos minutos.

Ligadas a un repositorio y a una rama concretos.



política de confianza del rol (extracto)

```
"Condition": {
  "StringEquals": {
    "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
    "token.actions.githubusercontent.com:sub": "repo:cic/infra:ref:refs/heads/main"
  }
}
```

Mínimo privilegio

El rol solo puede desplegar las pilas de este sistema, no administrar la cuenta.

Fuente: adaptado de AWS IAM User Guide, "OIDC federation" y GitHub Docs, "Configuring OpenID Connect in AWS"

CIERRE

Lo que queda de la Parte 3

Del commit a producción por un camino repetible: Git, validación, pruebas, entornos y seguridad.

1



Git

Todo cambio es un commit revisado

2



Validación

Fallar rápido, antes de gastar

3



Pruebas

Pirámide: muchas rápidas, pocas lentas

4



Entornos

Una plantilla, tres cuentas, aprobación al final

5



Seguridad

Credenciales temporales y mínimo privilegio



Lo que queda de la Parte 3

Un cambio de infraestructura llega a producción por un camino repetible: Git, validación, pruebas, aprobación.

Si algo falla, se vuelve atrás con una alarma o con un commit; si alguien cambia algo a mano, GitOps lo corrige.

Fuente: síntesis de la Parte 3; AWS Well-Architected Framework y AWS Prescriptive Guidance

ENCUADRE

La Unidad 2 en un mapa

Cinco semanas, cinco preguntas de ingeniería; la última explica cómo construir las otras cuatro con código.

S07



Cómputo

¿Dónde corre mi carga?

Instancias, tallas, precios, escalado

S08



Datos y respaldo

¿Dónde viven y se protegen mis datos?

Multi-AZ, réplicas, backup, RTO y RPO

S09



Identidad avanzada

¿Quién puede hacer qué?

Roles, federación, mínimo privilegio

S10



Alta disponibilidad

¿Cómo sigo en pie y crezco?

Balanceador, Auto Scaling, alarmas

S11



Automatización

¿Cómo lo construyo igual siempre?

Plantillas, pipelines, deriva



Cinco capas de una misma arquitectura

Cómputo, datos, acceso y resiliencia son las piezas; la automatización es el hilo que las construye igual cada vez.

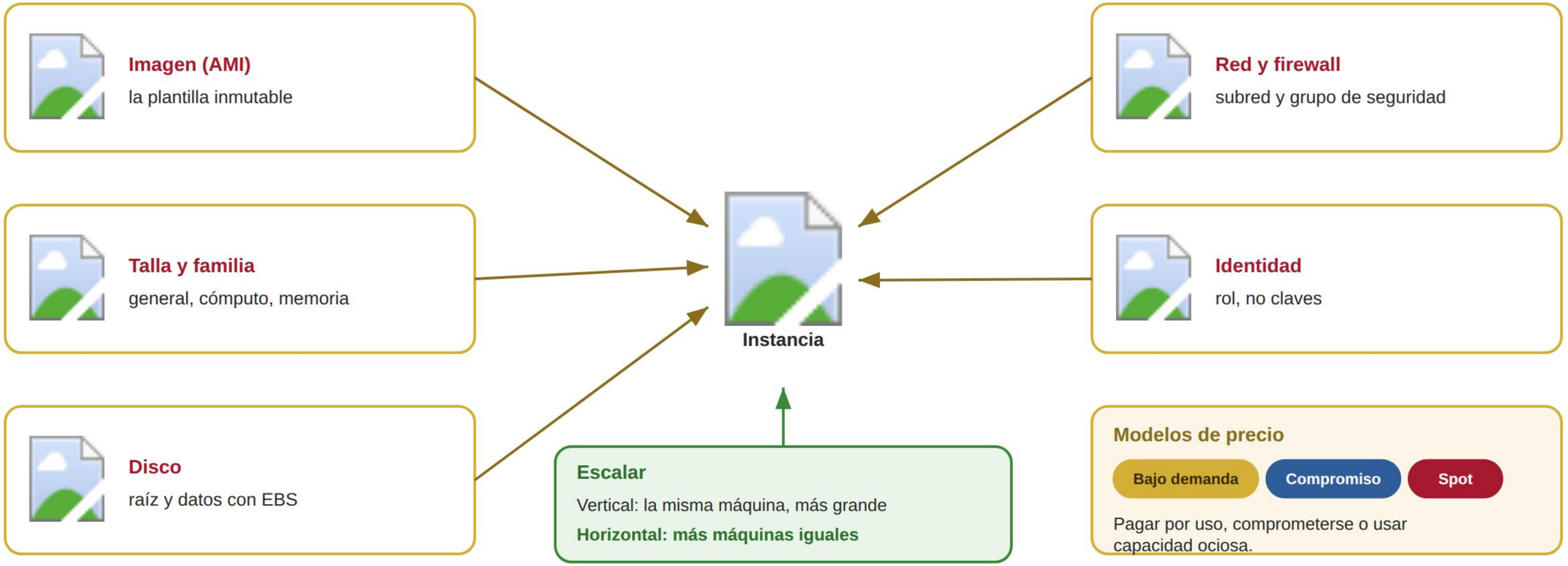
La semana 11 no agrega una pieza nueva: cambia la forma de armar todas las anteriores.

Fuente: calendario del curso 2026-2; AWS Academy Cloud Architecting, módulos 5, 6, 9, 10 y 11

REPASO U2

Semana 7: la instancia y sus decisiones

Una instancia es una imagen, una talla, un disco, una red y una identidad; todo lo demás es combinación.

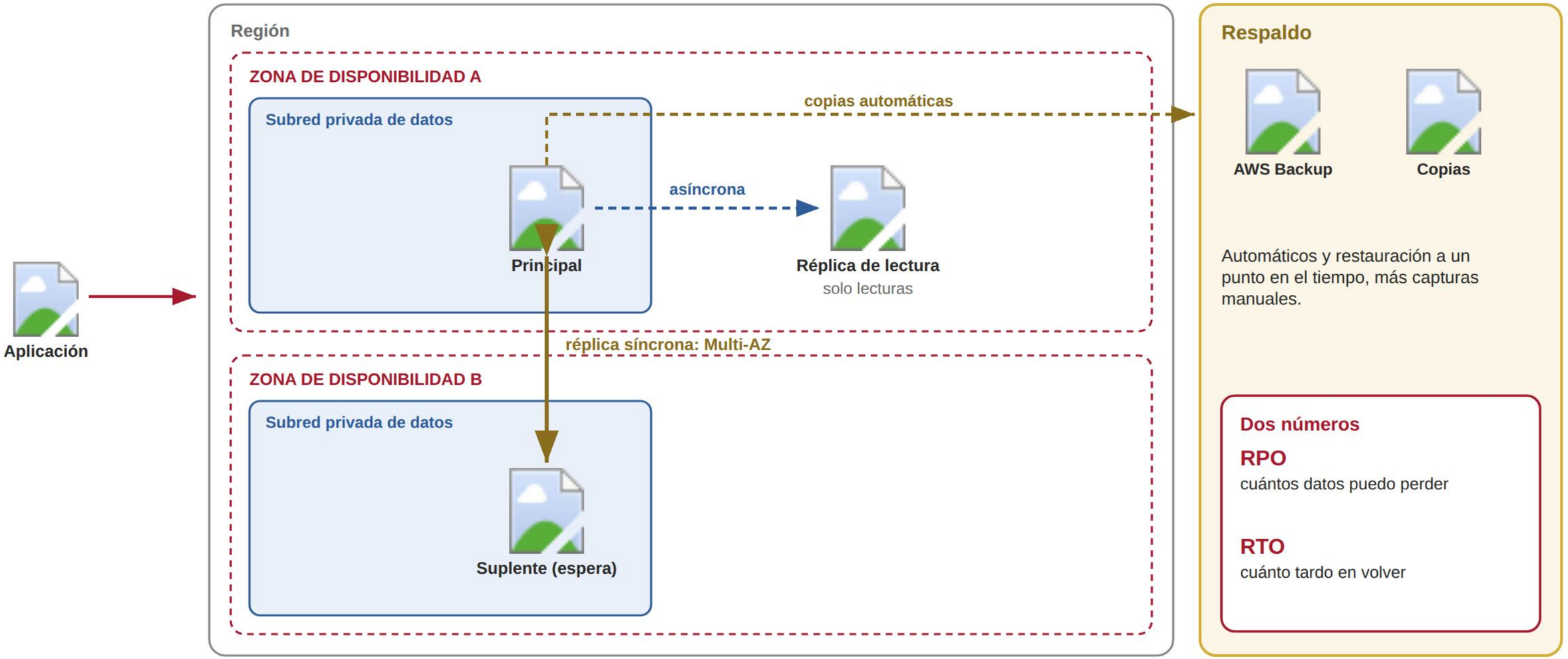


Casi todas estas decisiones son propiedades de un recurso: por eso caben en una plantilla.

Fuente: adaptado de AWS Academy Cloud Architecting, módulo 5 (capa de cómputo), y presentación de la Semana 7

Semana 8: datos disponibles y respaldados

Multi-AZ protege de una falla de zona; los respaldos protegen de errores humanos y corrupción.



Fuente: adaptado de AWS Academy Cloud Architecting, módulo 6 (capa de base de datos), y presentación de la Semana 8

Semana 9: identidad y mínimo privilegio

Los permisos efectivos son la intersección de varias capas; las credenciales deben ser temporales.

Capas que limitan los permisos

SCP de la organización (techo para toda la cuenta)

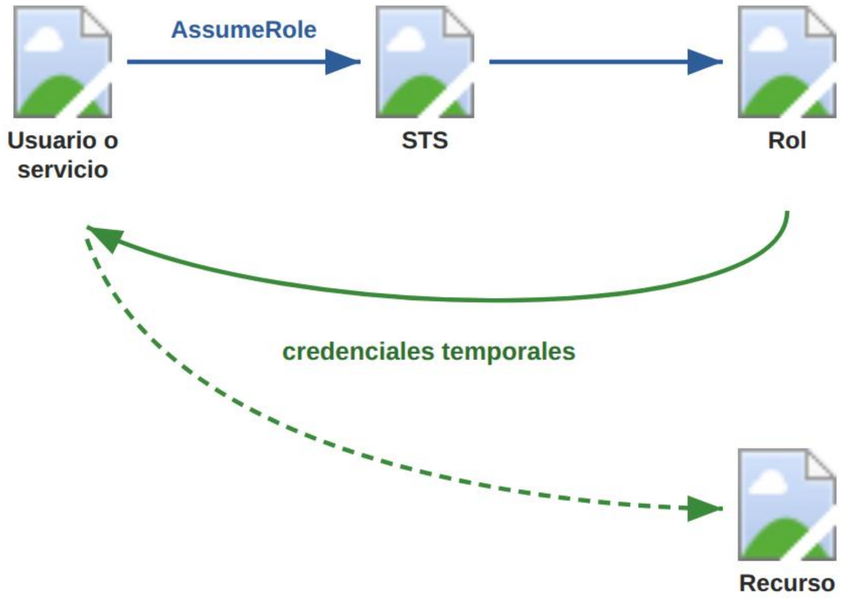
Límite de permisos del rol (permission boundary)

Política de identidad del rol

Permisos efectivos: solo lo que las tres capas permiten a la vez.

Un Deny explícito siempre gana sobre cualquier Allow.

Rol: credenciales temporales en lugar de claves



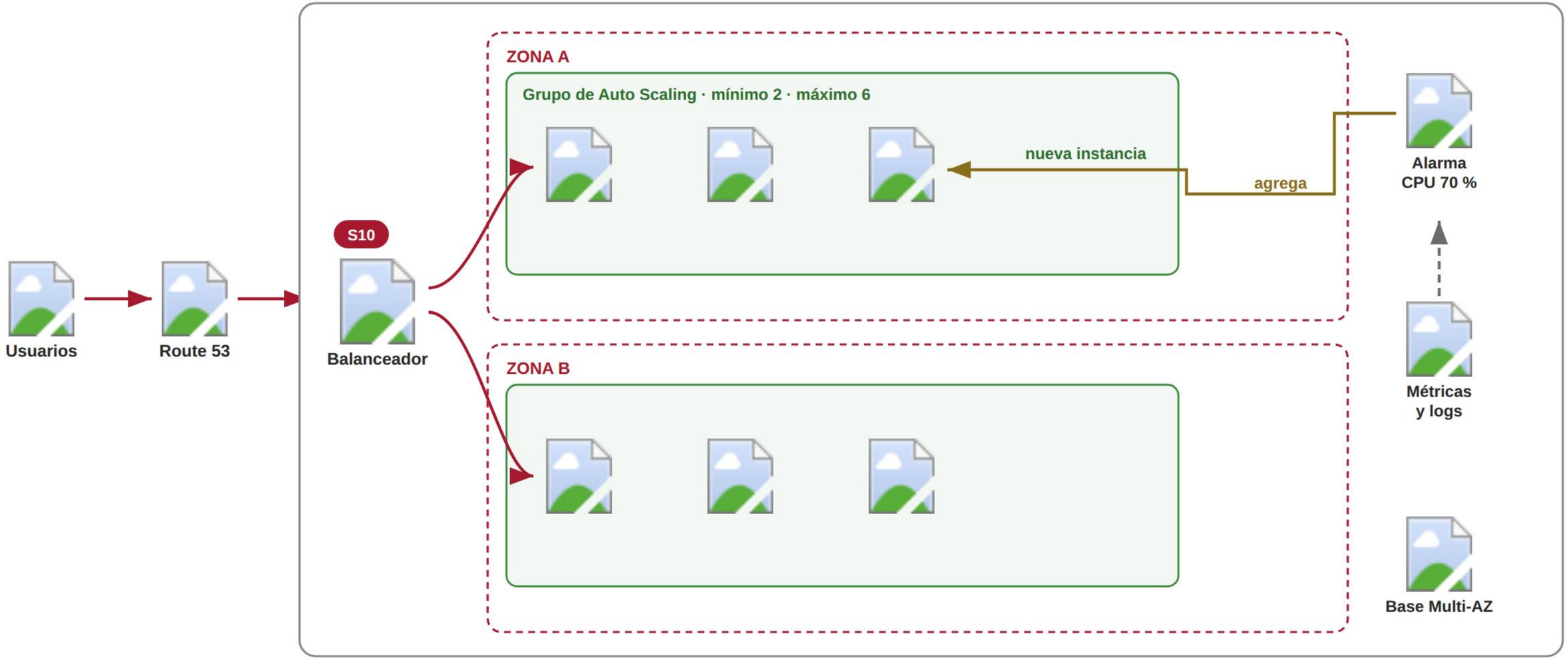
También en la semana 9

- Federación y SSO
- Cognito
- Multi-cuenta
- Gobernanza y costos

Fuente: adaptado de AWS IAM User Guide (evaluación de políticas, roles y STS) y AWS Academy Cloud Architecting, módulo 9

Semana 10: resistir y crecer solos

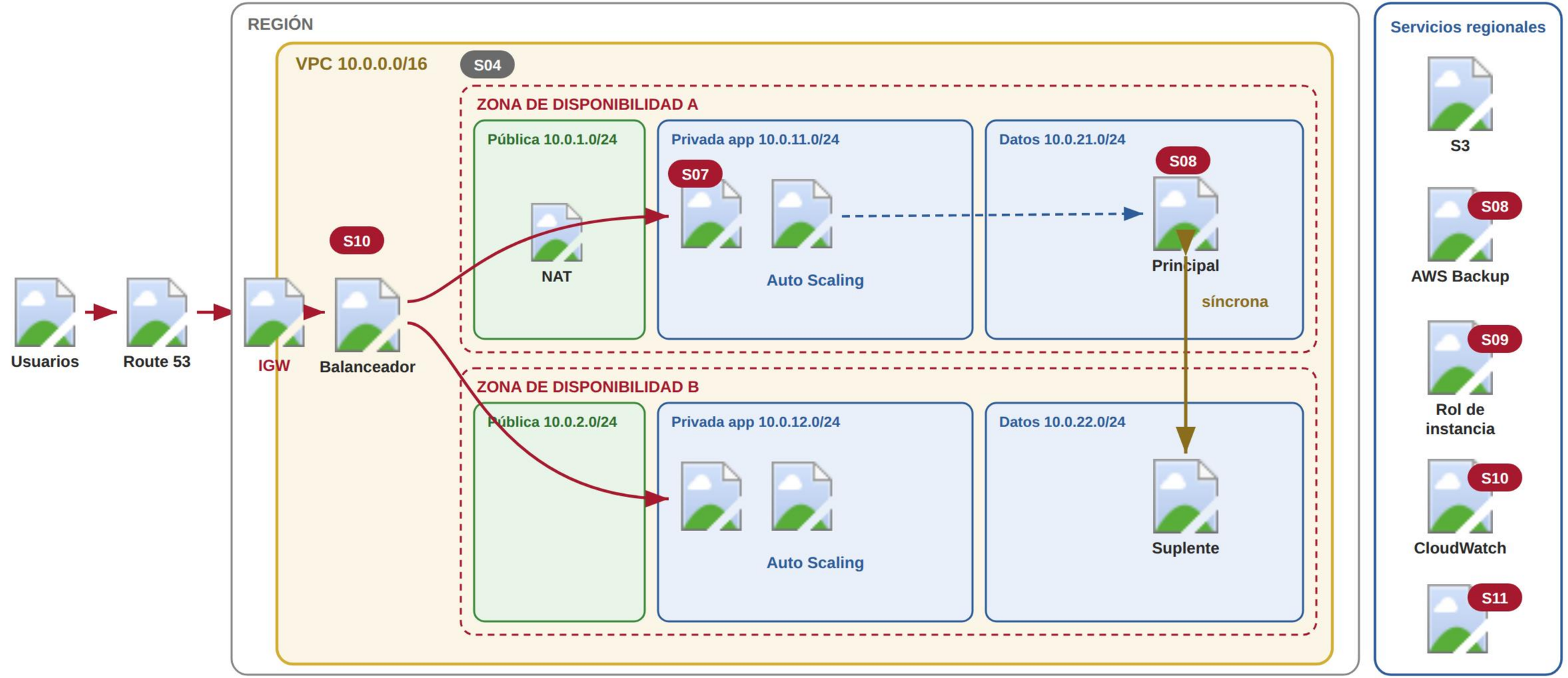
El balanceador reparte, Auto Scaling ajusta la cantidad y las alarmas deciden cuándo.



Fuente: adaptado de AWS Academy Cloud Architecting, módulo 10 y presentación de la Semana 10

Arquitectura de referencia de la CIC

Cómputo, datos, identidad y resiliencia en una sola arquitectura: lo que desplegaremos con una plantilla.



Fuente: adaptado de AWS Architecture Center y AWS Well-Architected Framework; caso CIC

La arquitectura como pilas en capas

Dividir por frecuencia de cambio permite desplegar y revertir cada capa sin tocar las demás.

red.yaml

Capa de red

- VPC y subredes
- Tablas de rutas
- IGW y NAT

Outputs: VpcId, SubredesPrivadas

datos.yaml

Capa de datos

- RDS Multi-AZ
- Grupo de seguridad BD
- Respaldos

Outputs: EndpointBD

computo.yaml

Capa de cómputo

- Balanceador
- Auto Scaling y lanzamiento
- Rol de instancia

Outputs: DnsBalanceador

observabilidad.yaml

Capa de observabilidad

- Alarmas de CPU y errores
- Panel de métricas
- Notificaciones SNS

Importa nombres de los recursos

Export de la red: VpcId y subredes, importados por cómputo

Export de datos: EndpointBD

Export de cómputo: DnsBalanceador

Cada capa es una pila con su propio ciclo de vida: la red casi no cambia; el cómputo cambia todos los días.

Alternativa: pilas anidadas (nested stacks) desde una plantilla raíz.

Fuente: adaptado de AWS CloudFormation User Guide (Export, ImportValue y pilas anidadas)

LABORATORIO

Laboratorio de IaC: qué se despliega

Una plantilla corta crea red mínima, servidor, bucket y salidas; luego se cambia y se vigila.

Qué vas a desplegar en el laboratorio

VPC de laboratorio

Subred pública



Servidor web



Grupo de seguridad

Parámetro: Entorno



Bucket



Todo desde una sola plantilla YAML, creada como una pila.

Salidas de la pila: dirección pública del servidor y nombre del bucket.

Parámetro: tamaño | **Etiquetas** | **Salidas útiles**

Cuatro fases

1. **Construir**
   Escribir y validar la plantilla
2. **Desplegar**
   Crear la pila y verificar
3. **Cambiar**
   Conjunto de cambios con otro tamaño
4. **Vigilar**
   Provocar y detectar deriva; eliminar

Fuente: elaboración propia para el curso (propuesta de laboratorio, 10 %); AWS CloudFormation User Guide

Laboratorio paso a paso

Seis comandos recorren todo el ciclo: validar, crear, consultar, cambiar, vigilar y limpiar.

1 Validar

```
aws cloudformation validate-template \
--template-body file://lab.yaml
```

2 Crear la pila

```
aws cloudformation create-stack \
--stack-name lab-iac \
--template-body file://lab.yaml \
--parameters ParameterKey=Entorno,ParameterValue=dev
```

3 Ver salidas

```
aws cloudformation describe-stacks \
--stack-name lab-iac
```

4 Cambiar

```
aws cloudformation create-change-set ...
aws cloudformation execute-change-set ...
```

5 Detectar deriva

```
aws cloudformation detect-stack-drift \
--stack-name lab-iac
```

6 Eliminar

```
aws cloudformation delete-stack \
--stack-name lab-iac
```


Fuente: adaptado de AWS CLI Command Reference (cloudformation); nombres de pila y archivo ilustrativos

LABORATORIO

Entregables del laboratorio

Cinco evidencias cuentan la historia completa: construir, desplegar, cambiar, vigilar y limpiar.

1. 
Plantilla YAML comentada
Con parámetros, recursos y salidas

2. 
Pila en CREATE_COMPLETE
Captura de la consola con las salidas

3. 
Conjunto de cambios
Vista previa y decisión sobre el reemplazo

4. 
Informe de deriva
Recurso modificado y valor esperado frente al real

5. 
Pila eliminada
Evidencia de limpieza: sin recursos residuales



Evaluación

10 % de la nota del curso

Los criterios y la rúbrica detallada los publica el docente junto con la guía del laboratorio.

Buenas prácticas esperadas:

- **Que la plantilla sea válida y legible**
- **Que el despliegue sea reproducible**
- **Que expliques cada decisión**
- **Que no queden recursos activos**

Antes de enviar: revisa que tu captura muestre el nombre de la pila y su estado.

Fuente: elaboración propia (propuesta de entregables; la rúbrica oficial la publica el docente)

LABORATORIO

Cuando la pila falla: cómo diagnosticar

El mensaje de error está en el primer evento fallido; casi siempre apunta a una de cuatro causas.

La pila termina en ROLLBACK_COMPLETE → **Abre la pestaña Eventos** → **Busca el primer evento CREATE_FAILED** → **Lee la causa, corrige y crea la pila de nuevo**

```
The image id does not exist
```

Causa: La AMI es de otra región.

Solución:
Usar un parámetro de SSM con la AMI vigente en cada región.

```
not authorized to perform: iam:CreateRole
```

Causa: Permisos insuficientes.

Solución:
En AWS Academy usa el rol y el perfil existentes, según la guía.

```
Bucket already exists
```

Causa: Los nombres de bucket son globales.

Solución:
Incluir la cuenta o el entorno en el nombre.

```
Limit exceeded
```

Causa: Se llegó a una cuota del servicio.

Solución:
Liberar recursos o solicitar un aumento.

Fuente: adaptado de AWS CloudFormation User Guide, "Troubleshooting CloudFormation"

PARCIAL 2

Orientación para el Parcial 2

Cinco semanas, tres habilidades: elegir, justificar y conectar las piezas de la arquitectura.

Parcial 2: 20 % de la nota, evaluado con rúbrica

S07 · Cómputo	S08 · Datos	S09 · Identidad	S10 · Resiliencia	S11 · Automatización
				
Elegir talla y familia	Multi-AZ frente a réplica	Roles y credenciales temporales	Balanceador y Auto Scaling	Declarativo e idempotente
Comparar modelos de precio	Respaldo, retención y PITR	Evaluación de políticas	Alarmas y métricas	Pila, cambios y deriva
Vertical frente a horizontal	Calcular RTO y RPO	Mínimo privilegio	Diseño multi-AZ	Pipeline y entornos

Cómo estudiar

1. Dibuja de memoria la arquitectura de referencia
2. Explica cada decisión: por qué esta pieza y no otra
3. Resuelve los escenarios de la siguiente diapositiva

Fuente: calendario del curso 2026-2 (Parcial 2, 20 %); contenidos de las semanas 7 a 11

Escenarios tipo: del problema a la pieza

Cada situación apunta a una decisión de la unidad; practica enlazarlas rápido.

Cae la zona A y la tienda se queda sin base de datos	RDS Multi-AZ: conmuta al suplente — S08
Alguien borró una tabla ayer por error	Respaldo con restauración a un punto en el tiempo — S08
El tráfico se triplica a fin de mes	Balanceador y Auto Scaling con alarmas — S10
Un desarrollador debe desplegar sin administrar la cuenta	Rol con mínimo privilegio y credenciales temporales — S09
Producción se comporta distinto que pruebas	Una plantilla, parámetros por entorno y pipeline — S11

Fuente: elaboración propia a partir de los contenidos de las semanas 7 a 11 (ejemplos de práctica)

CIERRE

Semana 11 en cuatro ideas y adelanto

La infraestructura como código se resume en declarar, revisar, entregar y vigilar.

1 Declarar



El estado deseado va en un archivo; el motor calcula los pasos.

2 Revisar



Conjuntos de cambios y pull requests antes de aplicar.

3 Entregar



Un pipeline lleva el cambio por dev, test y prod.

4 Vigilar



Deriva detectada y corregida; reversión disponible.

Adelanto: Semana 12

Caché y distribución de contenido en la nube



CloudFront



ElastiCache



Route 53

- **CloudFront: red de borde**
- **ElastiCache: caché en memoria**
- **Route 53: políticas de enrutamiento**

La arquitectura de hoy se acelera acercando los datos al usuario.

Trae tu arquitectura de referencia

Fuente: síntesis de la Semana 11; calendario del curso 2026-2 (Semana 12: caché y distribución de contenido)